

Sistema de Gestão de Escalas — PCCE

Apostila do Desenvolvedor

Guia completo de onboarding para desenvolvedores que estão
chegando agora

Setembro de 2026

Sumário

Como usar esta apostila

O que você vai encontrar

Como ler

Convenções tipográficas

Um aviso honesto sobre este projeto

Parte I — O terreno

1. O que é o sistema

1.1 O problema que ele resolve

1.2 Quem usa

1.3 Os módulos, em uma frase cada

1.4 O tamanho do problema

1.5 Por que "PCCE" aparece no código

2. Glossário do domínio

2.1 Vocabulário institucional

2.2 Vocabulário técnico do projeto

2.3 Vocabulário de assinatura digital

2.4 As siglas de achado

3. A stack, explicada para quem nunca viu

3.1 O quadro geral

3.2 Se você vem de outro ecossistema

3.3 TypeScript

3.4 Svelte 5 e as runes

3.5 SvelteKit

3.6 Cloudflare: Pages, Workers, D1 e R2

3.7 Drizzle ORM

3.8 Zod

3.9 Tailwind CSS e Skeleton UI

3.10 Vitest e Playwright

4. Montando o ambiente

4.1 Pré-requisitos

4.2 Clonar e instalar

4.3 Variáveis de ambiente

4.4 Criar o banco local

4.5 Subir o servidor

4.6 Testes ponta a ponta (opcional no primeiro dia)

4.7 Antes de abrir um PR

4.8 Problemas comuns no primeiro dia

Parte II — Como o sistema é construído

5. Mapa do repositório

5.1 A raiz

5.2 Dentro de `src/`

5.3 As quatro regras de "onde colocar o arquivo novo"

5.4 Como achar o código de uma tela

6. O ciclo de vida de uma requisição

6.1 A cadeia de hooks

6.2 Camada de CSRF

6.3 Camada de autenticação

6.4 Camada de segurança

6.5 Depois dos hooks: os três destinos

6.6 O caminho completo, com um exemplo

7. O banco de dados

7.1 D1, na prática

7.2 Onde as coisas vivem

7.3 O par obrigatório: schema + migração

7.4 Escrevendo uma migração

7.5 A herança que você precisa conhecer: unidade referenciada por NOME

7.6 Por que unidade não se exclui (a história)

7.7 A camada `$lib/db`

7.8 Datas: a armadilha que já mordeu três vezes

7.9 As tabelas, por grupo

8. Contratos: erros, validação e fetch

8.1 Erros de API — sempre por `$lib/server/api`

8.2 Autorização nos helpers

8.3 Validação de body com Zod

8.4 Fetch no cliente — sempre por `$lib/api-fetch`

8.5 Cabeçalhos de resposta

9. Autenticação, sessão e onboarding

9.1 O fluxo de login por senha

9.2 Por que o pepper importa

9.3 Login por certificado digital A3

9.4 Uma senha, duas identidades

9.5 Onboarding: duas fases, nesta ordem

9.6 Recuperação e limites

10. Autorização: a regra que define este projeto

10.1 O que é uma "operação material"

10.2 Por que não existe um `autorizar()` único

10.3 A regra do recurso, não só do usuário

10.4 Os helpers obrigatórios

10.5 A matriz de papéis

10.6 O complemento executável

Parte III — As regras inegociáveis

11. Entrada validada no servidor

11.1 O problema

11.2 As cinco formas aceitas de limitar

11.3 O corolário prático

- 11.4 A baseline, e por que ela está vazia
- 11.5 Um caso que vale estudar: a trava que não travava
- 11.6 Ausência declarada

12. Duplicação: extrair antes de comentar

- 12.1 A regra
- 12.2 Por que comentar não resolve
- 12.3 O guard
- 12.4 O corolário: quando NÃO extrair

13. LGPD, dado pessoal e minimização

- 13.1 O que é dado sensível aqui
- 13.2 Cifra em repouso e índice cego
- 13.3 Minimização por projeção
- 13.4 Quem alcança o forense
- 13.5 Retenção e direitos do titular
- 13.6 A régua para código novo

14. Assinatura digital e documentos com valor jurídico

- 14.1 As três modalidades
- 14.2 O fluxo de dois tempos
- 14.3 A intenção de assinatura
- 14.4 O ciclo comum e o portão
- 14.5 Verificação e validação pública
- 14.6 As flags de política
- 14.7 Golden antes de refatorar

15. Observabilidade e auditoria

- 15.1 Dois registros, dois propósitos
- 15.2 O fio que liga tudo: `request_id`
- 15.3 Como registrar
- 15.4 O que auditar

Parte IV — Os módulos, por dentro

16. Escalas ordinárias

- 16.1 O que o módulo faz
- 16.2 Onde está
- 16.3 O modelo
- 16.4 As regras que moram em `server/escalas/`
- 16.5 A assinatura da escala
- 16.6 O que costuma quebrar

17. Escala extra: operações e GISE

- 17.1 O vocabulário, primeiro
- 17.2 O modelo
- 17.3 Regras de exclusão e desativação
- 17.4 Configuração em cascata
- 17.5 O fluxo do membro
- 17.6 Rascunho local
- 17.7 Presença: o portão

18. Produtividade

- 18.1 A ideia
- 18.2 O modelo do formulário
- 18.3 Indicadores e metas
- 18.4 O que entra no painel
- 18.5 Como cada resposta é somada
- 18.6 Eixo, ordem e exportação

19. Plano operacional e custeio

- 19.1 O que difere da escala extra
- 19.2 A regra de custeio
- 19.3 De onde vêm os 100 km
- 19.4 O parecer do decreto
- 19.5 A distância é medida
- 19.6 Congelamentos
- 19.7 As listas do plano
- 19.8 O PDF e as pendências

20. Cadastro: servidores, solicitações e unidades

- 20.1 A ficha com dois poderes
- 20.2 As regras do cadastro, e o porquê de cada uma
- 20.3 Um padrão para copiar: o efeito mora fora da action
- 20.4 Unidades
- 20.5 Sincronização com a planilha

21. Consoles administrativos

Parte V — Trabalhando no projeto

22. Padrões de código

- 22.1 Svelte 5 — o resumo executável
- 22.2 SvelteKit — server-first
- 22.3 O guia visual
- 22.4 A armadilha do Skeleton: `transform` não se anula com `translate-*`
- 22.5 Onde a regra deve morar
- 22.6 Documentação de código

23. Testes

- 23.1 Os dois níveis
- 23.2 O que testar em cada nível
- 23.3 O spec que se escreve sozinho
- 23.4 O fluxo A3 em CI, sem token físico
- 23.5 Escrevendo um teste unitário
- 23.6 Testes manuais

24. CI, guards e deploy

- 24.1 O que o CI roda
- 24.2 Duas decisões do workflow que valem entender
- 24.3 Deploy
- 24.4 O endpoint destrutivo

25. Sua primeira contribuição

- 25.1 Antes de escrever código
- 25.2 O caminho completo, na ordem

25.3 Checklist de PR

25.4 Como pedir ajuda de forma eficiente

26. Exercícios guiados

27. Troubleshooting e FAQ

27.1 Ambiente

27.2 Assinatura

27.3 Perguntas frequentes de quem chega

Apêndices

Apêndice A — Inventário de rotas

A.1 Páginas

A.2 Endpoints por grupo

Apêndice B — Tabelas do banco

Apêndice C — Variáveis e bindings

C.1 Bindings Cloudflare (`wrangler.toml`)

C.2 Variáveis principais

Apêndice D — Comandos

Apêndice E — Onde ler mais

Palavra final

Como usar esta apostila

Esta apostila foi escrita para uma pessoa específica: a desenvolvedora ou o desenvolvedor **júnior que nunca viu este projeto**, acabou de receber acesso ao repositório e precisa, em poucos dias, entender o que o sistema faz, como ele está montado e como fazer a primeira alteração sem quebrar nada.

Ela não substitui a documentação viva do repositório — `README.md`, `DEPLOY.md`, `TESTING.md` e `CLAUDE.md` continuam sendo a fonte oficial e são atualizados no mesmo PR que muda o comportamento. O que esta apostila faz é **ensinar**: ela explica os conceitos antes de citar o arquivo, mostra o porquê antes da regra e sempre indica onde a informação vive no repositório para que você continue a leitura na fonte.

O que você vai encontrar

A apostila tem cinco partes e cinco apêndices:

- **Parte I — O terreno.** O que o sistema faz, para quem, com que vocabulário, sobre qual tecnologia e como montar o ambiente na sua máquina.
- **Parte II — Como o sistema é construído.** Estrutura de pastas, ciclo de vida de uma requisição, banco de dados, contratos de API, autenticação e autorização.
- **Parte III — As regras inegociáveis.** Validação de entrada, combate à duplicação, LGPD, assinatura digital e observabilidade. São as regras que o CI verifica e que já custaram bugs reais ao projeto.
- **Parte IV — Os módulos, por dentro.** Escalas, escala extra (GISE), produtividade, plano operacional, cadastro de servidores e consoles administrativos.
- **Parte V — Trabalhando no projeto.** Padrões de código, testes, CI/CD, o passo a passo da sua primeira contribuição, exercícios e solução de problemas.
- **Apêndices.** Inventários de consulta rápida: rotas, tabelas, variáveis de ambiente, comandos e mapa da documentação.

Como ler

Se você tem **um dia**, leia os capítulos 1, 3, 4, 5 e 6. Isso basta para rodar o projeto e entender o que acontece quando alguém abre uma tela.

Se você tem **uma semana**, leia tudo até o capítulo 15 e faça os exercícios do capítulo 26 na ordem. Deixe a Parte IV (módulos) para consulta: você vai ler o capítulo do módulo que a sua primeira tarefa tocar.

Se você já **está com uma tarefa em mãos**, vá direto ao capítulo 25 ("Sua primeira contribuição") e volte aos capítulos citados por ele conforme a necessidade.

Convenções tipográficas

- `caminho/de/arquivo.ts` — caminho relativo à raiz do repositório.
- **Negrito** — termo do domínio sendo definido, ou a parte da frase que carrega a decisão.
- Blocos com fundo cinza são código, comandos de terminal ou trechos reais do repositório.

- Os quadros iniciados por "⚠️", "✅" e "💡" destacam, respectivamente, uma armadilha conhecida, o jeito correto e uma dica de produtividade.

Um aviso honesto sobre este projeto

Este não é um CRUD. É um sistema que produz **documentos com valor jurídico** — escalas de serviço assinadas digitalmente com certificado ICP-Brasil, termos de presença que embasam pagamento de diária, relatórios de produtividade que viram política pública. Um bug aqui não gera só uma tela errada: pode gerar um documento assinado com conteúdo incorreto, um pagamento indevido ou a exposição de dado pessoal sensível de um servidor público.

Por isso o repositório é cheio de regras que, à primeira vista, parecem excesso de zelo: guards no CI, testes de golden byte a byte, comentários longos explicando decisões. Cada uma dessas regras nasceu de um problema real, e a apostila conta a história sempre que ela ajuda a entender a regra. **Não remova uma proteção sem entender o incidente que a criou.**

Parte I — O terreno

1. O que é o sistema

1.1 O problema que ele resolve

A Polícia Civil do Estado do Ceará (PCCE) organiza o trabalho dos seus servidores em **escalas**: quem trabalha, em qual unidade, em qual dia e horário. Antes deste sistema, esse controle vivia em planilhas do Google — uma por unidade, mantida à mão, impressa, assinada com caneta e digitalizada.

Isso funcionava até três perguntas aparecerem ao mesmo tempo:

1. **Como provar que aquele documento é autêntico?** Um PDF digitalizado com uma assinatura à caneta não prova quem assinou nem quando.
2. **Como pagar corretamente uma operação extraordinária?** Quando um grupo de servidores é escalado fora do expediente ou se desloca para outra cidade, o valor devido depende da hora, do dia, da distância e do cargo — e a conta era feita à mão.
3. **Como saber se a operação deu resultado?** Cada unidade produzia um relatório em formato próprio, e consolidar isso significava recomeçar do zero a cada mês.

O sistema responde às três: ele gera as escalas, produz os PDFs com layout oficial, coleta a **assinatura digital** (inclusive com certificado ICP-Brasil), registra a **presença** com selfie e GPS, calcula o **custeio** de cada operação e consolida os **indicadores de produtividade** em painéis.

1.2 Quem usa

O sistema tem cinco perfis de usuário, e quase toda regra de autorização se explica por eles:

Perfil	Quem é	O que faz
Super Admin	Responsável técnico do sistema	Tudo do Admin Geral, mais: promover administradores, gerenciar unidades, configurar a política de assinatura, ler os consoles de auditoria e baixar o material forense
Admin Geral	Coordenação estadual	Opera o sistema inteiro: cria escalas, aprova solicitações de cadastro, monta operações e planos, decide atos de RH
Admin de seccional	Chefia de uma seccional	Gerencia as escalas da sua seccional, solicita correções cadastrais dos servidores dela, informa a linha de base dos indicadores
Admin de unidade	Chefia de uma delegacia	O mesmo, escopado à sua unidade
Policial	Servidor comum	Vê as próprias escalas, confirma presença nas operações, preenche relatórios e assina os documentos que lhe cabem

Além desses, há papéis calculados dinamicamente dentro de uma operação — **supervisor**, **assessor/SEINT** e **membro** — que não são colunas de perfil, e sim consequência de estar (ou não) numa equipe daquela operação. Um policial comum pode ser supervisor de uma GISE e membro comum de outra.

1.3 Os módulos, em uma frase cada

Módulo	Rota principal	O que é
Escalas	/escalas	Escalas ordinárias: plantão, expediente e finais de semana (FDS). Gera PDF, coleta assinatura, valida publicamente
Escala extra	/gise	Operações extraordinárias (GISE, CRAJUBAR, EDGE...): equipes, presença com selfie e GPS, relatórios operacionais
Produtividade	/produtividade	Painel que consolida os relatórios da escala extra em indicadores, metas e gráficos
Plano operacional	/gise/planos	Operações com deslocamento : calcula diária ou hora extra por equipe e emite o plano em PDF
Cadastro	/policiais , /unidades	Servidores e hierarquia de unidades; solicitações de correção e atos de RH
Assinatura e validação	/conf-ass , /validar	Política de assinatura e o portal público que verifica um documento assinado
Consoles	/auditoria , /painel	Trilha forense, logs técnicos, LGPD e visão gerencial

1.4 O tamanho do problema

Números do repositório no momento em que esta apostila foi escrita — eles servem para calibrar a expectativa, não para decorar:

Medida	Valor aproximado
Linhas de TypeScript e Svelte em <code>src/</code>	152.000
Arquivos <code>.ts</code>	644
Componentes <code>.svelte</code>	179
Páginas (rotas com <code>+page.svelte</code>)	39
Endpoints de API (<code>+server.ts</code>)	71
Tabelas no banco	53
Migrações SQL versionadas	77
Specs de teste ponta a ponta (Playwright)	40+

⚠ Não tente ler o repositório inteiro. Ninguém no time leu. A forma correta de trabalhar aqui é entrar pela rota: abrir a tela no navegador, achar a pasta correspondente em `src/routes/`, ler o `+page.server.ts` dela e seguir os imports. O capítulo 6 ensina exatamente esse caminho.

1.5 Por que "PCCE" aparece no código

O sistema é institucional. Isso aparece em três lugares que confundem quem chega:

- **Nomes em português.** Funções, tabelas e variáveis são em português (`buscarEscala`, `gise_presencas`, `verificarPermissaoEscala`). É deliberado: o domínio é escrito em português na lei e nas portarias, e traduzir

"seccional" para "sectional" só cria uma camada de tradução mental.

- **Siglas do domínio.** DPC (Delegado de Polícia Civil), OIP (Oficial de Investigação Policial), SEINT (equipe de inteligência), NUP (número único de protocolo). O capítulo 2 tem o glossário completo.
- **Referências a normas.** Comentários citam a Lei 14.063/2020, a MP 2.200-2, o Decreto nº 36.182/2024 e a LGPD. Elas não são decoração: quando o código diz "art. 13", há um teto legal implementado ali.

2. Glossário do domínio

Leia este capítulo uma vez e volte a ele sempre que uma palavra aparecer sem sentido. Ele está dividido em vocabulário **institucional** (o que a corporação chama assim) e vocabulário **técnico do projeto** (o que o time chama assim).

2.1 Vocabulário institucional

Termo	Significado
Unidade	Qualquer lotação: departamento, seccional ou delegacia. Elas formam uma hierarquia de três níveis
Seccional	Unidade intermediária que agrupa delegacias de uma região
Delegacia (DP)	Unidade operacional de ponta
Lotação	A unidade onde o servidor está alocado. No banco, é gravada pelo nome da unidade, não por chave estrangeira (ver §7.5)
Plantão	Escala de 24h (ou turno) com equipes se revezando
Expediente	Escala do horário administrativo, de segunda a sexta
FDS	Escala de fim de semana; tem regras próprias de assinatura
Escala extra	Serviço extraordinário, fora da jornada ordinária, remunerado como hora extra
GISE	Grupo de Intervenção e Suporte Especializado. Historicamente foi a primeira operação extraordinária do sistema, e o nome ficou como prefixo de rotas e tabelas — hoje é uma operação entre várias
Operação	O catálogo de uma ação extraordinária (GISE, CRAJUBAR, EDGE): nome, sigla, ciclo, tipos de equipe e formulários próprios
Plano operacional	Documento de uma operação com deslocamento : equipes que saem da sua cidade para cumprir mandados em outra
DPC / OIP	Delegado de Polícia Civil / Oficial de Investigação Policial. São os dois cargos que ocupam as vagas ("slots") de uma equipe
SEINT	Equipe de inteligência de uma operação (o outro tipo é "operacional")
Classe	Faixa dentro do cargo (1ª, 2ª, 3ª, especial; A, B, C, D). Define o valor da hora extra
Diária	Verba paga quando a missão extrapola a jornada e envolve deslocamento relevante
Hora extra	Verba paga por hora trabalhada fora do expediente, quando não couber diária
NUP	Número Único de Protocolo — identifica o processo administrativo de um ato de RH
Portaria	Documento que formaliza movimentação, afastamento ou desvinculação de servidor. É anexada em PDF ao pedido
Linha de base	O valor de partida de um indicador numa unidade. É o denominador das metas percentuais
Termo de presença	Comprovante assinado de que o servidor esteve na operação. Embasa o pagamento

2.2 Vocabulário técnico do projeto

Termo	Significado
Operação material	Qualquer handler que muda estado : <code>POST</code> , <code>PUT</code> , <code>PATCH</code> , <code>DELETE</code> de API e toda form action. Toda operação material precisa recusar alguém no servidor
Portão (gate)	Função que decide se a operação prossegue. Ex.: <code>carregarEscalaParaAssinatura</code>
Guard	Script que roda no CI e reprova o build quando uma regra do projeto é violada (<code>npm run guard:autorizacao</code> , <code>guard:duplicacao</code> , <code>guard:entrada</code> , <code>guard:achados</code>)
Golden	Arquivo de referência byte a byte (PDF, e-mail). Se a saída mudar sem intenção, o teste falha
Intenção de assinatura	Registro que amarra um PDF preparado ao documento, ao assinante e a um único uso
Manifesto	Página final do PDF assinado, com os dados forenses do ato (CPF, IP, GPS, selfie, hash)
Selo institucional	Certificado do próprio sistema, usado na assinatura avançada (quando não há e-CPF do titular)
Blob de respostas	O JSON com as respostas de um formulário de produtividade, gravado em uma coluna
Slot	Vaga de uma equipe (uma vaga de DPC, uma de OIP)
Achado	Item de auditoria interna, identificado por sigla (<code>FLW-ESC-002</code> , <code>SEC-08</code> , <code>A3</code>). Ver §2.4
Baseline	Lista de exceções aceitas por um guard. Existe para dívida assumida conscientemente, não para silenciar o guard

2.3 Vocabulário de assinatura digital

Este bloco é o que mais assusta quem chega. Leia agora em diagonal; o capítulo 14 explica cada item em detalhe.

Termo	Significado
ICP-Brasil	A infraestrutura de chaves públicas oficial brasileira. Um certificado emitido dentro dela tem presunção legal de autenticidade
e-CPF	Certificado digital de pessoa física dentro da ICP-Brasil
A1 / A3	Formatos de certificado: A1 fica em arquivo no computador; A3 fica em token/cartão físico, e a chave privada nunca sai do hardware
PAdES	Padrão de assinatura embutida em PDF
CAdES	Padrão de assinatura sobre dados quaisquer; é o que vai dentro do PAdES
OCSP	Protocolo que pergunta à autoridade certificadora se um certificado foi revogado
TSA / carimbo de tempo	Autoridade que atesta <i>quando</i> a assinatura foi feita, de forma oponível a terceiros
Assinatura qualificada	Feita com e-CPF ICP-Brasil. É o nível mais forte
Assinatura avançada	Feita com 2FA + selfie + GPS + selo institucional. Vale juridicamente (Lei 14.063/2020) com outro peso probatório
WebPKI / SERPRO Desktop	Os dois softwares que fazem a ponte entre o navegador e o token A3 do usuário
Passkey / WebAuthn	Chave de assinatura registrada no celular do titular, que prova controle exclusivo do aparelho

2.4 As siglas de achado

Espalhados pelo código você vai encontrar comentários como:

```
// FLW-ESC-002: membro de outra escala virava editável por ID  
// SEC-08: escapeLike sozinho não fecha o buraco
```

São referências a **auditorias internas** já encerradas. Os relatórios não vivem mais no working tree — foram removidos para enxugar o repositório — mas continuam preservados no histórico do Git e catalogados em `docs/HISTORICO.md`, com o comando `git show` que recupera cada um.

A regra do projeto é: **toda sigla citada no código tem de ter onde ser lida**. O CI verifica isso com `npm run guard:achados`. Quando você encontrar uma sigla e quiser entender o contexto, comece por `docs/HISTORICO.md`.



Algumas siglas (`M-6`, `M-8`, `M-10`, `I-6`) são declaradas como **sem documento**: o relatório de origem nunca foi commitado, e o comentário ao lado da sigla é o registro inteiro. Isso está escrito no catálogo de propósito — a diferença entre "órfã e ninguém sabe" e "órfã, sabemos, e está escrito" é o que torna a rastreabilidade honesta.

3. A stack, explicada para quem nunca viu

Este capítulo apresenta cada tecnologia em três blocos: **o que é, por que este projeto usa e o mínimo que você precisa saber para ler o código**. Se você já domina alguma, pule.

3.1 O quadro geral

Camada	Tecnologia	Versão principal
Linguagem	TypeScript	5
Meta-framework	SvelteKit	2
UI	Svelte (com runes)	5
Estilo	Tailwind CSS + Skeleton UI	4 / 4
Execução	Cloudflare Pages / Workers (edge)	—
Banco	Cloudflare D1 (SQLite serverless)	—
Arquivos	Cloudflare R2 (compatível com S3)	—
ORM	Drizzle ORM	0.4x
Validação	Zod	4
Assinatura	pdf-lib, @signpdf, node-forge, web-pki	—
Documentos	jsPDF, ExcelJS, docx	—
Reconhecimento facial	@vladmandic/face-api (TensorFlow.js)	1
Monitoramento	Sentry	10
Testes	Vitest (unitário) + Playwright (E2E)	4 / 1
Build	Vite	8

3.2 Se você vem de outro ecossistema

Uma tabela de tradução, para acelerar a leitura. À esquerda, o que você provavelmente já usou; à direita, o equivalente aqui.

Você conhece	Aqui é	Observação
Vue 3 (<code>ref</code> , <code>computed</code> , <code>watch</code>)	Svelte 5 (<code>\$state</code> , <code>\$derived</code> , <code>\$effect</code>)	Conceitualmente igual: valor reativo, valor derivado, efeito colateral
Pinia / Vuex	Arquivos <code>.svelte.ts</code> com <code>\$state</code> exportado	Não há store global obrigatória; a maior parte do estado vem do servidor
Vue Router	Roteamento por pastas do SvelteKit	A pasta é a URL
Nuxt	SvelteKit	Mesmo papel: SSR, rotas, endpoints, build
Express + controllers	<code>+page.server.ts</code> e <code>+server.ts</code>	O arquivo da rota já é o controller
Sequelize	Drizzle ORM	Bem mais fino: você escreve algo próximo de SQL, com tipos
MySQL	SQLite (Cloudflare D1)	Sem <code>ALTER</code> completo; migrações são escritas à mão
<code>express-validator</code> / <code>Joi</code>	Zod	Schemas em <code>src/lib/schemas/</code>
PM2 / container Node	Cloudflare Workers	Sem processo longo, sem <code>fs</code> , sem <code>setInterval</code> global
Multer + disco	Cloudflare R2	Objetos em bucket, acessados por binding
Jest	Vitest	API quase idêntica
Cypress	Playwright	Specs em <code>e2e/</code>

⚠ A diferença que mais pega quem vem do Node tradicional: no Cloudflare Workers **não existe processo que fica de pé**. Cada requisição roda isolada, pode cair num data center diferente, e não há memória compartilhada entre elas. Isso significa: nada de cache em variável de módulo esperando persistir, nada de `setInterval`, nada de conexão de banco reaproveitada. Tudo que precisa sobreviver entre requisições vive no D1, no R2 ou no cache de edge.

3.3 TypeScript

O que é. JavaScript com tipos verificados antes de rodar.

Por que aqui. O sistema manipula dinheiro, datas e documentos assinados. Um `undefined` inesperado num cálculo de diária vira pagamento errado. O projeto roda em modo `strict`, e o CI reprova qualquer erro de tipo (`npx svelte-check --threshold error`).

O mínimo para ler o código:

```
// Tipo de objeto
type Escala = { id: number; lotacao: string; mes: number };

// União: só estes valores são aceitos
type Tipo = 'plantao' | 'expediente' | 'fds';

// Genérico: T é decidido por quem chama
```



```

async function apiFetch<T>(url: string): Promise<T> { /* ... */ }

// `as const` congela o objeto e transforma os valores em tipos literais
export const ErrorCode = { VALIDATION: 'validation' } as const;

```

O padrão `X | Response` aparece muito no projeto — é a forma de dizer "ou deu certo e devolvo o dado, ou deu errado e devolvo a resposta HTTP pronta":

```

const usuario = requireAuth(locals);
if (usuario instanceof Response) return usuario; // 401 já formatado
// daqui para baixo, `usuario` é UsuarioLogado – o TypeScript sabe disso

```

3.4 Svelte 5 e as runes

O que é. Svelte é um framework de UI que compila os componentes: em vez de levar um runtime grande para o navegador, ele gera JavaScript que mexe direto no DOM. Um componente é um arquivo `.svelte` com três seções: script, markup e estilo.

Por que aqui. Bundle pequeno importa quando o usuário abre o sistema num celular em delegacia com conexão ruim. Além disso, o modelo de reatividade do Svelte 5 (runes) é explícito e fácil de auditar.

As runes. São funções especiais reconhecidas pelo compilador. O projeto **obriga** o uso delas — a sintaxe antiga do Svelte 4 está proibida em código novo:

```

<script lang="ts">
  // props do componente (antes: export let)
  let { titulo, escala, onSalvar } = $props();

  // estado reativo local (antes: let comum)
  let nome = $state('');

  // valor derivado, recalculado sozinho (antes: $:)
  let nomeMaiusculo = $derived(nome.toUpperCase());

  // efeito colateral quando algo que ele lê muda (antes: onMount/$:)
  $effect(() => {
    document.title = titulo;
  });
</script>

<input bind:value={nome} />
<p>{nomeMaiusculo}</p>

```

❌ Nunca em código novo	✅ Use
<code>export let titulo</code>	<code>let { titulo } = \$props()</code>
<code>let nome = ''</code> (querendo reatividade)	<code>let nome = \$state('')</code>
<code>\$: upper = nome.toUpperCase()</code>	<code>let upper = \$derived(...)</code>
<code>writable()</code> de <code>svelte/store</code>	<code>\$state</code> em arquivo <code>.svelte.ts</code>
<code>onMount()</code> para lógica reativa	<code>\$effect()</code>
<code><slot /></code>	<code>snippets ({#snippet} / {@render})</code>

Composables. Lógica reativa reutilizável mora em arquivos com extensão dupla `.svelte.ts` — a extensão é o que autoriza o compilador a processar runes fora de um componente:

```
// src/lib/composables/useContador.svelte.ts
export function useContador(inicial = 0) {
  let valor = $state(inicial);
  const dobro = $derived(valor * 2);
  return {
    // getter: mantém a reatividade ao atravessar a fronteira
    get valor() { return valor; },
    get dobro() { return dobro; },
    incrementar() { valor++; }
  };
}
```

⚠️ Repare nos **getters**. Se você devolvesse `{ valor }` direto, entregaria uma cópia do número no instante da chamada e a reatividade morreria ali. Esse é o erro nº 1 de quem escreve o primeiro composable.

📖 Antes de implementar qualquer coisa em Svelte, o `CLAUDE.md` manda consultar a documentação oficial: <https://svelte.dev/docs/svelte/overview> e <https://svelte.dev/docs/kit/introduction>. O framework mudou bastante na versão 5, e respostas antigas de fórum ainda ensinam a sintaxe proibida.

3.5 SvelteKit

O que é. O framework em volta do Svelte: roteamento, renderização no servidor, endpoints de API, build e adaptadores de hospedagem.

Roteamento por pastas. A pasta é a URL. Dentro dela, o nome do arquivo diz o papel:

Arquivo	Papel
+page.svelte	A tela (roda no servidor na primeira carga e depois no navegador)
+page.server.ts	<code>load()</code> (busca os dados) e <code>actions</code> (recebe formulários). Só roda no servidor
+server.ts	Endpoint de API: exporta <code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>PATCH</code> , <code>DELETE</code>
+layout.svelte	Moldura compartilhada por todas as rotas abaixo
+layout.server.ts	<code>load()</code> compartilhado
+error.svelte	Tela de erro
[id] (pasta)	Parâmetro dinâmico: <code>/escalas/[id]</code> → <code>/escalas/42</code>
components/ (pasta)	Peças privadas da rota. O <code></code> a mantém fora do roteador

As duas formas de mudar dados. Este é o conceito central do SvelteKit, e o projeto usa as duas:

1. **Form action** — o formulário HTML envia `FormData` para uma action no `+page.server.ts`. Funciona até sem JavaScript, e é o caminho preferido para formulários de tela.
2. **Endpoint de API** — o navegador chama `/api/...` com JSON. É o caminho para tudo que não é um formulário simples: downloads, fluxos de assinatura, buscas incrementais.

```
// +page.server.ts – as duas metades de uma rota
export const load: PageServerLoad = async ({ locals }) => {
  return { policiais: await buscarPoliciais(getDB(locals.platform)) };
};

export const actions: Actions = {
  salvar: async ({ request, locals }) => {
    const data = await request.formData();
    // ... valida, autoriza, grava
    return { sucesso: true };
  }
};
```

locals e platform. Dois objetos que você verá em toda assinatura de função no servidor:

- `locals` — o que os hooks colocaram ali para esta requisição. Neste projeto: `locals.usuario` (a sessão resolvida) e `locals.requestId` (o identificador de correlação).
- `platform` — os recursos da Cloudflare vinculados ao Worker: `platform.env.escalas_db` (banco) e `platform.env.escalas_docs` (bucket).

3.6 Cloudflare: Pages, Workers, D1 e R2

O que é. O sistema não roda num servidor com um Node ligado. Ele é publicado como **Cloudflare Pages** e executado como **Worker**: um código que a Cloudflare roda no data center mais próximo do usuário, sob demanda.

Consequências práticas (todas já causaram bug neste projeto ou em projetos parecidos):

Regra	Motivo
Sem <code>fs</code> , sem <code>path</code> , sem <code>Buffer</code>	Não há sistema de arquivos. Use R2 e <code>Uint8Array</code>
Sem estado em memória entre requisições	Cada requisição pode cair em outro data center
Sem <code>setInterval</code> / cron dentro do app	Tarefas periódicas entram por webhook autenticado (ex.: <code>/api/webhook/limpeza-retencao</code>)
Tempo de CPU limitado	Por isso o PBKDF2 usa 100k iterações — é o teto do runtime
Cache de edge é por PoP	Invalidar o cache local não alcança outro data center. Por isso sessão em cache não vale para métodos que mudam estado

D1 é o banco: SQLite gerenciado pela Cloudflare. Em desenvolvimento, o Wrangler simula tudo na sua máquina (`.wrangler/state/v3/d1/`), então você não precisa de conta Cloudflare para programar.

R2 é o armazenamento de objetos (equivalente ao S3): guarda PDFs assinados, selfies das presenças e portarias anexadas.

Os dois chegam ao código como **bindings** declarados em `wrangler.toml`:

```
[[d1_databases]]
binding = "escalas_db"

[[r2_buckets]]
binding = "escalas_docs"
```

E são lidos assim:

```
import { getDB, getR2 } from '$lib/db';
const db = getDB(locals.platform); // cliente Drizzle sobre o D1
const r2 = getR2(locals.platform); // bucket
```

3.7 Drizzle ORM

O que é. Uma camada fina entre o TypeScript e o SQL. Diferente do Sequelize ou do Prisma, o Drizzle não esconde o SQL — ele te dá o SQL com tipos.

```
import { eq, and } from 'drizzle-orm';
import { escalas } from '$lib/server/schema';

const linha = await db
  .select()
  .from(escalas)
  .where(and(eq(escalas.id, id), eq(escalas.ativo, 1)))
  .get(); // .get() = uma linha; .all() = várias
```

O **schema** (`src/lib/server/schema.ts`, 53 tabelas) descreve as tabelas em TypeScript:

```
export const escalas = sqliteTable('escalas', {
  id: integer('id').primaryKey({ autoIncrement: true }),
  lotacao: text('lotacao').notNull(),
  mes: integer('mes').notNull()
});
```

⚠ **A armadilha número um do projeto inteiro:** editar o `schema.ts` **não cria coluna nenhuma**. Ele só muda o tipo que o TypeScript enxerga. Uma coluna que existe só no schema compila, passa no `npm run check`, passa no lint — e explode no primeiro `SELECT` em produção. Toda mudança de banco exige **o par:** `schema` + um arquivo novo em `migrations/`. Ver §7.3.

3.8 Zod

O **que é**. Biblioteca de validação de dados que também gera o tipo TypeScript correspondente.

```
import { z } from 'zod';

export const escalaSchema = z.object({
  lotacao: z.string().min(1).max(120),
  mes: z.number().int().min(1).max(12),
  tipo: z.enum(['plantao', 'expediente', 'fds'])
});

type Escala = z.infer<typeof escalaSchema>; // o tipo sai do schema, de graça
```

Os schemas vivem em `src/lib/schemas/`. Nas rotas de API, o uso obrigatório é via `validateBody` (§8.3). O ponto de Zod aqui não é conveniência: é que `.max()`, faixa e enum vêm **de graça** e fecham a porta para entradas absurdas (`mes = 99`) que já geraram bug real neste projeto.

3.9 Tailwind CSS e Skeleton UI

Tailwind é CSS por classes utilitárias: em vez de escrever uma folha de estilo, você compõe `class="flex items-center gap-2 rounded-xl px-4 py-2"`.

Skeleton UI é uma biblioteca de componentes construída sobre o Tailwind (modal, combobox, switch...).

O projeto tem uma regra rígida: **cores sempre pelos canais do tema** (`primary`, `secondary`, `tertiary`, `success`, `warning`, `error`, `surface`), nunca cores cruas da paleta do Tailwind (`text-red-500`). Os tokens vivem em `src/theme.css` e `src/app.css`. O capítulo 22 traz o guia visual completo — e ele é mais importante do que parece, porque metade dos bugs visuais do projeto veio de gente inventando um valor fora da escala.

3.10 Vitest e Playwright

Vitest roda os testes unitários, em ambiente `node` (sem DOM). Todo `*.test.ts` mora numa pasta `__tests__` junto do código testado.

Playwright roda os testes ponta a ponta: sobe a aplicação de verdade num navegador de verdade e clica nas telas. Os specs ficam em `e2e/`.

⚠ Componente `.svelte` não tem teste unitário neste projeto, e isso é decisão registrada. O Vitest roda sem DOM; quem exercita componente é o Playwright. A consequência prática é uma regra de arquitetura: **se uma regra precisa de teste, ela sai do `.svelte` e vai para um `.ts` puro.** Precisar montar um componente para testar algo é o sinal de que esse algo está no arquivo errado.

4. Montando o ambiente

4.1 Pré-requisitos

- **Node.js 22 ou superior** (o `package.json` exige, e o CI usa 22).
- **npm** (vem com o Node).
- **Wrangler** — a CLI da Cloudflare: `npm install -g wrangler@latest`.
- Opcional, para E2E: navegador do Playwright (instruções no §4.6).

Você **não** precisa de conta Cloudflare para desenvolver: o Wrangler simula D1 e R2 localmente.

4.2 Clonar e instalar

```
git clone <url-do-repositorio> escalas
cd escalas
npm install
```

4.3 Variáveis de ambiente

O Wrangler lê os segredos locais de um arquivo `.dev.vars` na raiz (que **não** vai para o Git). Comece copiando o exemplo comentado:

```
cp .env.example .dev.vars
```

O mínimo para o sistema subir:

```
SYNC_TOKEN=<openssl rand -hex 32>
RESET_TOKEN=<outro openssl rand -hex 32, diferente do primeiro>
```

⚠ Gere os tokens de verdade. Os webhooks recusam `SYNC_TOKEN` com menos de 32 caracteres — é uma trava proposital contra segredo fraco em produção. Um placeholder do tipo `token-de-dev` faz quatro specs de E2E falharem com 401, e a falha *parece* bug do sistema quando é só configuração.

Opcional, se quiser testar envio de e-mail (2FA, primeiro acesso):

```
RESEND_API_KEY=re_...
```

```
RESEND_FROM_EMAIL=onboarding@resend.dev
```

📖 A lista completa e comentada de variáveis está em `.env.example` — ela é a **fonte autoritativa**, não a tabela do README. Os tipos correspondentes estão em `src/app.d.ts`. O apêndice C resume as principais.

4.4 Criar o banco local

```
npm run db:migrate
```

Isso aplica todas as migrações SQL na instância local do D1. Rode este comando também toda vez que der `git pull` e vierem migrações novas.

4.5 Subir o servidor

```
npm run dev
```

Acesse <http://localhost:5173>.

Fluxo de trabalho recomendado — três terminais:

```
# Terminal 1: servidor
npm run dev

# Terminal 2: type-check contínuo (fortemente recomendado)
npm run check:watch

# Terminal 3: testes unitários em watch
npm run test:watch
```

4.6 Testes ponta a ponta (opcional no primeiro dia)

```
npx playwright install --with-deps chromium # só uma vez
npm run test:e2e
```

O comando faz build e sobe a aplicação sozinho; o `global-setup` aplica migrações pendentes e semeia as fixtures. Não é preciso preparar o banco à mão.

⚠️ **O E2E roda contra o seu D1 local, não contra um banco limpo.** O setup limpa apenas o que a própria suíte cria (a faixa de id 99xxx). Se você andou usando o app, seus dados continuam lá — os specs são escritos para tolerar isso. Se algum falhar por causa de dado alheio, é bug do spec, não do seu banco.

4.7 Antes de abrir um PR

```
npm run lint:fix    # corrige o que dá para corrigir sozinho
npm run format      # Prettier
npm run check       # type-check (o mesmo do CI)
npm run test        # testes unitários
```

O apêndice D lista todos os comandos disponíveis.

4.8 Problemas comuns no primeiro dia

Sintoma	Causa provável	Solução
<code>npm run dev</code> falha com erro de binding D1/R2	O banco local ainda não existe	<code>npm run db:migrate</code>
Erro de tipo em <code>.svelte</code> que o editor não mostra	O <code>svelte-check</code> é mais rigoroso que o servidor de linguagem	Rode <code>npm run check</code> e confie nele
403 de CSRF ao chamar a API	<code>fetch</code> cru sem os headers CSRF	Use <code>apiFetch</code> de <code>\$lib/api-fetch</code> (§8.4)
Sessão expira logo após o login	Relógio da máquina fora de sincronia	Sincronize o relógio (NTP); o D1 usa UTC
Quatro specs de E2E falhando com 401	<code>SYNC_TOKEN</code> curto demais	Gere com <code>openssl rand -hex 32</code>
Tela em branco depois de mexer num componente	Erro de runtime do Svelte	Abra o console do navegador; o erro está lá, não no terminal

Parte II — Como o sistema é construído

5. Mapa do repositório

5.1 A raiz

```

escalas/
├─ src/           # todo o código da aplicação
├─ migrations/    # migrações SQL, escritas à mão, numeradas
├─ scripts/       # utilitários Node e os guards do CI
├─ e2e/           # testes ponta a ponta (Playwright)
├─ docs/          # documentação complementar (e esta apostila)
├─ static/        # arquivos servidos como estão (modelos de face-api)
├─ wrangler.toml  # bindings da Cloudflare (D1, R2)
├─ svelte.config.js, vite.config.ts, tsconfig.json
├─ README.md      # visão geral, setup, arquitetura, módulos
├─ DEPLOY.md      # runbook de produção
├─ TESTING.md     # roteiro manual de exceção
└─ CLAUDE.md      # as diretrizes obrigatórias de código

```

 **CLAUDE.md** é curto e é a leitura mais rentável do repositório. Ele condensa as regras que o CI verifica. Leia-o inteiro no seu primeiro dia — a Parte III desta apostila é, em boa medida, a explicação didática dele.

5.2 Dentro de `src/`

```

src/
├─ routes/        # rotas (páginas + APIs). A pasta É a URL
│   └─ api/       # endpoints REST
│   └─ _components/ # regras de navegação compartilhadas por VÁRIAS rotas
│   └─ +layout.svelte # moldura: sidebar, tema, toasts
│   └─ +layout.server.ts # load global (usuário, flags, papel)
│   └─ ...        # uma pasta por tela
├─ lib/
│   └─ components/ # componentes Svelte reutilizáveis
│   └─ composables/ # lógica reativa reutilizável (.svelte.ts)
│   └─ server/     # BACKEND – nunca importar no cliente
│   └─ db/         # camada de acesso a dados (queries tipadas)
│   └─ schemas/    # schemas Zod
│   └─ crypto/     # primitivas (hash, token, timing-safe, AES)
│   └─ utils/      # utilidades puras (sem barrel)
│   └─ gise/, planos/, produtividade/, diarias/ # regras puras de domínio
│   └─ api-fetch.ts # fetch da API interna (obrigatório no cliente)

```

```

|   └─ auth.ts           # tipos e helpers de RBAC
|   └─ ...
└─ hooks.server.ts       # middleware de toda requisição
└─ app.d.ts              # tipos globais (bindings, App.Locals)
└─ app.css, theme.css    # estilos globais e tokens do tema
└─ app.html

```

💡 O prefixo `$lib` nos imports é um atalho do SvelteKit para `src/lib`. `$lib/db` é `src/lib/db.ts`; `$lib/server/api` é `src/lib/server/api.ts`.

⚠️ **Tudo dentro de `src/lib/server/` é backend.** O SvelteKit **impede** que esses arquivos sejam importados por código de cliente — se você tentar, o build falha. É proposital: é o que garante que um segredo ou uma query não vaze para o navegador.

5.3 As quatro regras de "onde colocar o arquivo novo"

Estas regras estão no `CLAUDE.md` e são verificadas (algumas pelo CI, outras por revisão). Elas existem porque o repositório já foi bagunçado o suficiente para doer.

Regra 1 — em `src/lib/server/` : raiz é infra transversal, subpasta é domínio.

A raiz só aceita o que é usado por vários domínios sem pertencer a nenhum: `api.ts`, `schema.ts`, `logger.ts`, `sentry.ts`, `request-context.ts`, `csp.ts`, `app-origin.ts`, `db-errors.ts`, `email.ts`, `r2-cleanup.ts`, `policial-permissao.ts`, `edge-cache.ts`, `form-data.ts`, `rate-limit-pesado.ts`.

Todo o resto vai para a subpasta do domínio — `assinatura/`, `auth/`, `escalas/`, `gise/`, `export/`, `sync/`, `termo/`, `planos/`, `policiais/`, `operacoes/` — junto com o seu `__tests__/`.

💡 Um nome de arquivo que só faz sentido com prefixo de domínio (`gise-algo.ts`, `escala-algo.ts`) é sinal de que ele pertence a uma subpasta. Até julho de 2026 essa pasta era plana, com 58 arquivos e cinco domínios misturados. Não a deixe voltar a ser.

Regra 2 — arquivo auxiliar de rota vai em `_components/`.

Componente, composável ou action que serve a **uma** rota mora em `_components/` (ou `_actions/`) dentro dela. O `_` é o que mantém o arquivo fora do roteador.

- Peça usada por **duas rotas irmãs** sobe para `$lib/components/`.
- Peça usada por uma rota e a sub-rota dela fica no `_components/` do diretório que contém as duas — é a pasta da **família**. Nesse caso, cada arquivo declara no cabeçalho quais telas o consomem. Pasta de família sem essa declaração é armadilha: parece privada e não é.
- Peça que estava no pai e só a filha usa **desce** para o `_components/` da filha.

`src/routes/_components/` (na raiz das rotas) é a exceção deliberada: regra de navegação consultada por várias rotas, em `.ts` puro e com teste — hoje `menu-visibilidade.ts` (o que a sidebar mostra) e `bem-vindo-cards.ts` (os quadros das telas de boas-vindas).

Regra 3 — todo teste mora em `__tests__/` junto do código testado.

`src/lib/gise/x.ts` é testado por `src/lib/gise/__tests__/x.test.ts`. Nunca ao lado do fonte. O CI reprova o contrário. Teste ponta a ponta é outra história: vai em `e2e/`.

Regra 4 — `$lib/utils/` não tem barrel.

Importe o módulo, não a pasta: `$lib/utils/datas`, `$lib/utils/formato`, `$lib/utils/pii`, `$lib/utils/download`, `$lib/utils/localStorage`. O ganho é o call site dizer de qual assunto a função veio. (`$lib/db` é a exceção documentada — lá o barrel existe e está justificado no próprio arquivo.)

5.4 Como achar o código de uma tela

Este é o roteiro que o time usa todo dia:

1. Abra a tela no navegador e olhe a URL: `/escalas/42`.
2. Traduza para pasta: `src/routes/escalas/[id]/`.
3. Abra o `+page.server.ts` — ele tem o `load` (de onde vêm os dados) e as `actions` (o que os formulários fazem). **É onde mora a lógica de negócio mais próxima do banco.**
4. Abra o `+page.svelte` — a tela.
5. Siga os imports: `$lib/db/...` para consultas, `$lib/server/<dominio>/...` para regras, `$lib/components/...` para peças de UI.

Para um endpoint, o caminho é o mesmo: `/api/escalas/42/download` → `src/routes/api/escalas/[id]/download/+server.ts`.

6. O ciclo de vida de uma requisição

Este capítulo responde à pergunta "o que acontece entre o clique e a tela". É o capítulo mais importante da apostila.

6.1 A cadeia de hooks

Todo pedido HTTP — página, endpoint, download — passa antes por `src/hooks.server.ts`. Lá, cinco camadas rodam **nesta ordem**, e a ordem é a parte que importa:

```
requisição
|
├─ 1. handleRequestContext → cria o requestId e o contexto de log
├─ 2. handleSentry          → embrulha para capturar exceções
├─ 3. handleCsrf            → double-submit cookie nas rotas /api que mudam estado
├─ 4. handleAuth            → resolve a sessão, impõe onboarding e termo
├─ 5. handleSecurity        → cabeçalhos de resposta, incluindo a CSP
|
└─ load() / action / endpoint da rota
```

Por que essa ordem:

- **1 vem primeiro** para que tudo adiante — inclusive uma falha nos próprios hooks — seja correlacionável por um `requestId`. É esse id que aparece para o usuário como código de rastreamento em erros 5xx, vira tag no Sentry e liga a trilha de auditoria aos logs técnicos.

- **3 vem antes de 4** de propósito: requisição forjada é recusada **sem custar uma consulta de sessão** ao banco. Inverter abriria caminho para gastar D1 com tráfego malicioso.

6.2 Camada de CSRF

O projeto usa o padrão **double-submit cookie**:

1. O servidor emite um cookie `csrf_token` com 256 bits aleatórios. Ele **não** é `httpOnly`, porque o JavaScript precisa lê-lo.
2. Toda requisição `POST / PUT / PATCH / DELETE` para `/api/*` precisa mandar o header `x-csrf-token` com o mesmo valor.
3. A comparação é *timing-safe* (a mesma primitiva usada em credenciais).

Rotas isentas: `/api/auth/login`, `/api/health` e `/api/webhook` — as duas últimas se autenticam por segredo próprio, e o login ainda não tem sessão.

Como o login é isento, existe uma **segunda camada** só para `/api/auth/`: a checagem de `Origin`. *Origem divergente é sempre recusada; origem ausente é recusada onde o `Origin` é a única camada. Isso fecha o ataque de forced login** (um site malicioso logar a vítima na conta do atacante).

✓ No cliente, você nunca monta esses headers à mão: use `apiFetch` (§8.4), que já os injeta.

6.3 Camada de autenticação

```
tem cookie session_token?
  | não → rota pública? segue sem usuário : redireciona /login (ou 401 na API)
  | sim
  | └─ tenta o CACHE DE EDGE da sessão (TTL curto)
  |     └─ método muda estado (POST/PUT/PATCH/DELETE)? TTL = 0, vai direto ao banco
  | └─ valida a sessão no D1 e, no MESMO batch, o aceite do termo vigente
  | └─ renova o cookie (sliding), exceto em rotas de poll
  | └─ portão 1: primeiro_acesso pendente → /alterar-senha
  | └─ portão 2: termo de uso não aceito → /aceitar-termo
  | └─ portão 3: módulo (Escalas/GISE) liberado para a conta?
  | └─ grava locals.usuario e segue
```

Três decisões vale a pena entender agora, porque explicam bugs que já aconteceram:

Sessão em cache não vale para quem muda. O cache de edge é por PoP (ponto de presença da Cloudflare), e nenhuma invalidação local alcança outro data center. Leitura pode estar um TTL atrasada; **ação, não**. Por isso `ttlCacheSessaoParaMetodo` devolve 0 para métodos que mudam estado.

O cookie desliza junto com o banco. A sessão é estendida no banco a cada uso, mas o `maxAge` do cookie só era reescrito no login. As duas metades discordavam, e quem trabalhasse além do TTL era deslogado pelo navegador com a sessão viva no banco — um logout duro no meio de uma cerimônia de assinatura. Hoje o cookie é reemitido a cada requisição **que não seja poll de fundo**: uma aba aberta renovando sozinha a cada 120 s fazia a sessão nunca expirar.

A ordem dos portões de onboarding é obrigatória. Enquanto `primeiro_acesso` está pendente, o portão do termo **não** é imposto — senão o usuário cairia num 403 sem saída, porque `/aceitar-termo` é inalcançável enquanto o portão da senha devolve todo mundo para `/alterar-senha`.

6.4 Camada de segurança

Toda resposta recebe cabeçalhos de segurança, entre eles:

Cabeçalho	Efeito
<code>X-Frame-Options: DENY</code>	Impede o site ser embutido em iframe (clickjacking)
<code>X-Content-Type-Options: nosniff</code>	Impede o navegador "adivinhar" o tipo do conteúdo
<code>Referrer-Policy: strict-origin-when-cross-origin</code>	Limita o que vaza no referer
<code>Permissions-Policy</code>	Libera câmera e geolocalização só para a própria origem
<code>Cross-Origin-Resource-Policy: same-origin</code>	Impede outros sites carregarem nossos recursos
<code>Cross-Origin-Opener-Policy: same-origin</code>	Isola o contexto de navegação
<code>Cross-Origin-Embedder-Policy: credentialless</code>	Isolamento sem quebrar recursos externos
CSP	Content Security Policy montada em <code>\$lib/server/csp</code>

Além disso, resposta autenticada que **não** define `Cache-Control` recebe `private, no-store` por padrão. Guarde esta frase: **definir um `Cache-Control` mais fraco numa rota é pior do que não definir nada**, porque tira a rota do default seguro. Há uma constante para isso, `CACHE_PRIVADO`, e o capítulo 13 conta por que ela existe.

6.5 Depois dos hooks: os três destinos

A) Página com `load()` — o SvelteKit chama o `load` do `+layout.server.ts` e o do `+page.server.ts`, junta o resultado e renderiza o `+page.svelte` com esses dados.

```
export const load: PageServerLoad = async ({ locals, params }) => {
  const db = getDB(locals.platform);
  const escala = await buscarEscala(db, Number(params.id));
  if (!escala) error(404, 'Escala não encontrada');
  // autorização acontece aqui, no servidor
  const { permitido } = await verificarPermissaoEscala(db, escala.id, escala.lotacao,
    locals.usuario!);
  if (!permitido) error(403, 'Sem permissão');
  return { escala };
};
```

Na tela, os dados chegam por `$props()`:

```
<script lang="ts">
  let { data } = $props();
```

```
</script>
<h1>{data.escala.lotacao}</h1>
```

B) Form action — o formulário aponta para uma action nomeada:

```
<form method="POST" action="/salvar" use:enhance>
```

```
export const actions: Actions = {
  salvar: async ({ request, locals }) => {
    const fd = await request.formData();
    // 1. autoriza 2. valida a entrada 3. grava 4. audita
    return { sucesso: true };
  }
};
```

⚠ Form action não usa o status HTTP para dizer que falhou. O `ActionResult` viaja em JSON sob HTTP 200 mesmo quando a action recusou. Isso importa ao escrever testes: quem procura 403 numa action nunca vai achar.

C) Endpoint de API — `+server.ts` exportando o método:

```
export const POST: RequestHandler = async ({ request, locals }) => {
  const usuario = requireAuth(locals);
  if (usuario instanceof Response) return usuario;

  const v = await validateBody(request, meuSchema);
  if (!v.ok) return v.response;

  // ... trabalho
  return json({ ok: true });
};
```

6.6 O caminho completo, com um exemplo

Um admin abre `/escalas/42` e clica em "Baixar PDF". O que acontece:

Passo	Onde	O que acontece
1	<code>hooks.server.ts</code>	<code>requestId</code> criado; sessão resolvida; <code>locals.usuario</code> preenchido
2	<code>routes/escalas/[id]/+page.server.ts</code>	<code>load</code> busca a escala, verifica permissão de leitura e devolve só o que a tela consome
3	<code>routes/escalas/[id]/+page.svelte</code>	Renderiza; o botão de download chama <code>apiFetchResponse</code>
4	<code>hooks.server.ts</code>	Nova requisição: CSRF (é GET, não exige), sessão resolvida de novo
5	<code>routes/api/escalas/[id]/download/+server.ts</code>	Verifica permissão de novo (o servidor nunca confia no que a tela já verificou), busca o PDF no R2
6	resposta	<code>Content-Disposition</code> montado com o helper (§8.5), <code>Cache-Control: private, no-store</code>

💡 Repare no passo 5: a autorização é refeita. Isso não é redundância burra — a tela e o endpoint são superfícies diferentes, e a tela pode estar aberta há meia hora com uma permissão que já foi revogada.

7. O banco de dados

7.1 D1, na prática

O D1 é SQLite. Isso traz três consequências que moldam o código:

1. **ALTER TABLE é limitado.** Mudar o tipo de uma coluna, adicionar uma constraint ou trocar uma FK exige o ritual completo: criar tabela nova → copiar dados → dropar a antiga → renomear.
2. **Não há tipos ricos.** Data e hora são texto; booleano é `0 / 1`.
3. **Chave estrangeira é aplicada de verdade.** O D1 respeita FK, e isso já salvou o sistema (§7.6).

7.2 Onde as coisas vivem

Arquivo	Papel
<code>src/lib/server/schema.ts</code>	O schema Drizzle: 53 tabelas. É o que o TypeScript enxerga
<code>migrations/00NN_descricao.sql</code>	O que realmente muda o banco . Escritas à mão
<code>src/lib/db.ts</code>	O barrel: a API que rotas e endpoints consomem
<code>src/lib/db/</code>	A implementação, por assunto: <code>core.ts</code> , <code>escalas.ts</code> , <code>policiais/</code> , <code>gise/</code> , <code>audit/</code> , <code>lgpd/</code> , <code>planos/</code> ...

7.3 O par obrigatório: schema + migração

⚠️ Esta é a regra que mais causa dor a quem chega. Editar `schema.ts` não cria coluna nenhuma. O TypeScript passa a acreditar que a coluna existe; o banco continua sem ela; o erro só aparece no primeiro `SELECT`, em runtime.

Toda alteração de banco é um par:

```
src/lib/server/schema.ts      ← o tipo
migrations/0075_minha_coluna.sql ← a mudança de verdade
```

E depois:

```
npm run db:migrate      # aplica localmente
```

⚠ **Não use `drizzle-kit generate`**. O `drizzle.config.ts` ainda aponta para o schema e as 12 primeiras migrações saíram dele, mas o gerador não produz o *rebuild* de tabela que o SQLite do D1 exige para quase todo `ALTER` real.

O que já rodou em cada ambiente é rastreado pela tabela `_migrations_aplicadas`, gravada pelo runner `scripts/migrate.ts`.

7.4 Escrevendo uma migração

Convenções observadas nos 77 arquivos existentes:

- Numeração sequencial de quatro dígitos, nome descritivo: `0074_municipios_regiao_metropolitana.sql`.
- Uma migração por mudança conceitual.
- Migração **não se edita depois de aplicada** em qualquer ambiente. Se errou, escreva a próxima.
- Índice único parcial é uma ferramenta usada e recomendada aqui. Exemplo real: `uq_plano_opcoes_padrao` é único sobre `(plano_id, tipo)` `WHERE padrao = 1`, o que impede duas abas do navegador deixarem duas opções marcadas como padrão — a arbitragem é do banco, não de um `SELECT` antes do `INSERT`.

7.5 A herança que você precisa conhecer: unidade referenciada por NOME

`policiais.lotacao` e `escalas.lotacao` guardam o **nome** da unidade, não uma chave estrangeira. É herança da planilha que originou o sistema, e tem duas consequências permanentes:

- **Renomear cascadeia.** `atualizarUnidade` propaga o nome novo para policiais e escalas na mesma operação. Um `UPDATE` direto no banco quebraria os vínculos silenciosamente.
- **Unidade não se exclui — só se desativa.** Não existe ação de excluir na interface nem função de `DELETE` na camada de dados. `definirUnidadeAtiva` marca `ativo = 0`.

7.6 Por que unidade não se exclui (a história)

Vale ler devagar, porque é o exemplo canônico do tipo de risco que este sistema corre.

`giseassinaturas_relatorios.seccional_id` referencia `unidades(id)`, e o D1 aplica FK de verdade. Com `ON DELETE CASCADE`, apagar uma unidade levava junto o **registro do ato de assinar** — assinante, CPF, selfie, IP, GPS, hash do arquivo e a chave do PDF no R2. O portal público `/validar` passava a responder "documento não encontrado" para um papel que alguém já tinha em mãos, **indistinguível de documento falso**. Escala e lotação, que ligam por nome e sem FK, ficavam órfãs sem erro nenhum.

Defesa em profundidade aplicada: a FK passou de `CASCADE` para `RESTRICT` (migração 0038). Hoje, mesmo um `DELETE` manual fora da aplicação é recusado pelo banco.

💡 Guarde o padrão: **a ausência da ação na interface é a primeira barreira; a constraint no banco é a segunda**. Quando você projetar algo destrutivo, pense nas duas.

7.7 A camada `$lib/db`

Rotas não montam SQL. Elas chamam funções da camada de dados:

```
import { getDB, buscarEscala, listarUnidades, auditar } from '$lib/db';
```

Dentro de `src/lib/db/core.ts` moram os helpers transversais. Três merecem destaque porque carregam armadilhas reais:

`getDB(platform)` — devolve o cliente Drizzle. É barato (só embrulha o binding), então cada handler chama o seu; não há conexão para reaproveitar em Workers. Ele **lança** se o binding não existir, em vez de devolver `undefined` — o erro aponta o problema real (Wrangler mal configurado) em vez de estourar longe dali.

`likeContains(coluna, termo)` — busca textual. Nunca escreva `like()` à mão:

```
// ✅ correto
.where(likeContains(policiais.nome, termo))
```

Dois motivos. O primeiro é *wildcard injection*: quem digita `100%` procura o texto `100%`, não "tudo que começa com 100" — por isso `escapeLike` escapa `%`, `_` e `\`. O segundo é um limite do D1: `SQLITE_MAX_LIKE_PATTERN_LENGTH` vale **50**, não os 50.000 do SQLite padrão. Um padrão maior não devolve zero linhas — devolve erro 500. E isso acontecia na vida real: todo `SearchableSelect` reescreve no campo o rótulo do item escolhido ("FULANO DE TAL — OIP Mat. 30010124", 52 caracteres) e esse eco dispara uma busca nova. Acima do limite, `likeContains` troca a forma para `instr(lower(col), lower(termo)) > 0`, que tem a mesma semântica. Truncar o padrão seria pior: devolveria um superconjunto em silêncio.

Timestamps. O formato de data/hora do SQLite e as duas convenções de fuso (UTC × Brasília) têm funções próprias, com o fuso no nome. Timestamp errado não lança exceção: apaga dado ou desliga rate-limit em silêncio.

7.8 Datas: a armadilha que já mordeu três vezes

O sistema é usado em UTC-3. Um `new Date().toISOString().slice(0, 10)` devolve a data **em UTC**, o que significa que entre 21h e a meia-noite o "hoje" do sistema vira amanhã. Isso já causou:

- calendário marcando o dia seguinte das 21h à meia-noite;
- o laço "dias do intervalo" trocando local por UTC em três lugares diferentes;
- `toISO` com duas convenções de mês (base 0 e base 1) gerando data de mês errado, **sem erro nenhum**.

✅ Use sempre `$lib/utils/datas: hoje(), MESES_PT, DIAS_SEMANA_CURTO, opcoesMeses()`. Antes de declarar uma constante "óbvia" de data no componente, verifique se ela já existe lá. E lembre: `MESES_PT` tem índice 0 = Janeiro (a base de `Date.getMonth()`); para mês 1–12 vindo do banco ou da URL, use `MESES_PT[mes - 1]`.

7.9 As tabelas, por grupo

O apêndice B tem a lista completa. Para orientação inicial, os grupos:

Grupo	Tabelas principais
Pessoas e acesso	<code>policiais</code> , <code>administradores</code> , <code>sessoes</code> , <code>aceites_termos</code> , <code>credenciais_webauthn</code>
Cadastro e RH	<code>unidades</code> , <code>policial_historico</code> , <code>cadastro_solicitacoes</code> , <code>policial_acao_solicitacoes</code>
Escalas	<code>escalas</code> , <code>escala_policiais</code> , <code>escala_documentos</code> , <code>escala_solicitacoesassinatura</code>
Operações extraordinárias	<code>operacoes</code> , <code>operacao_linha_base</code> , <code>gise_escalas</code> , <code>gise_seccionais</code> , <code>gise_equipes</code> , <code>gise_membros</code> , <code>gise_presencas</code> , <code>gise_documentos</code>
Produtividade	<code>gise_modelo_formulario</code> , <code>gise_respostas_formulario</code> , <code>gise_assinaturas_relatorios</code>
Plano operacional	<code>planos_operacionais</code> , <code>plano_equipes</code> , <code>plano_equipe_membros</code> , <code>plano_opcoes</code> , <code>custo_parametros</code> , <code>municipios</code> , <code>distancias_municipios</code>
Assinatura	<code>assinatura_intencoes</code> , <code>assinatura_reauth</code> , <code>gise_presenca_termos</code>
Registro e conformidade	<code>audit_log</code> , <code>app_log</code> , <code>audit_pendencias</code> , <code>audit_checkpoints</code> , <code>lgpd_incidentes</code> , <code>lgpd_solicitacoes</code>
Infra	<code>configuracoes</code> , <code>login_attempts</code> , <code>recovery_attempts</code> , <code>webhook_nonces</code> , <code>r2_pendencias</code>

8. Contratos: erros, validação e fetch

Este capítulo cobre quatro contratos obrigatórios. "Obrigatório" aqui não é figura de linguagem: o CI reprova o PR que não os seguir.

8.1 Erros de API — sempre por `$lib/server/api`

 **Nunca** escreva isto em rota nova:

```
return json({ error: 'Escala não encontrada' }, { status: 404 });
```

 Escreva isto:

```
import { notFound } from '$lib/server/api';
return notFound('Escala');
```

O motivo é o front-end. O corpo padronizado é `{ error, status, errorType?, errorId? }`, e o `errorType` é um **enum fechado** (`ErrorCode`), não string livre. Com ele, a tela decide o comportamento sem depender da mensagem em português (que pode mudar a qualquer momento): `validation` realça o campo, `csrf` recarrega para pegar token novo, `auth_required` manda para o login, `rate_limit` esconde o formulário.

A tabela de referência:

Cenário	Chamada	ErrorCode	HTTP
Body inválido / Zod	<code>badRequest('msg')</code>	VALIDATION	400
Sem sessão / token expirado	<code>unauthorized()</code>	AUTH_REQUIRED	401
Autenticado, mas sem permissão	<code>forbidden('msg')</code>	FORBIDDEN	403
Token CSRF inválido	<code>apiError('...', 403, ErrorCode.CSRF)</code>	CSRF	403
Recurso inexistente	<code>notFound('Escala')</code>	NOT_FOUND	404
Conflito de estado (já assinado)	<code>conflict('msg')</code>	CONFLICT	409
Rate-limit ultrapassado	<code>rateLimited()</code>	RATE_LIMIT	429
Falha externa (e-mail, OCSP)	<code>apiError('msg', 502, ErrorCode.UPSTREAM)</code>	UPSTREAM	502
5xx inesperado	<code>serverError('contexto', err)</code>	INTERNAL	500

⚠ Nunca passe string livre como `errorType`. Precisa de categoria nova? Adicione ao enum em `src/lib/server/api.ts`.

`serverError` merece um parágrafo próprio. Ele gera um `errorId` de 8 caracteres, registra o erro completo no log e **devolve só o id ao usuário**. A mensagem técnica (que traz SQL, parâmetros e stack) nunca chega à tela. O usuário lê "reporte o código de rastreamento", informa o id, e o operador correlaciona com o Sentry e com a tabela `app_log` em um passo.

8.2 Autorização nos helpers

```
const usuario = requireAuth(locals);          // 401 se não houver sessão
if (usuario instanceof Response) return usuario;

const admin = requireAdmin(locals);           // 403 se não for Admin Geral
if (admin instanceof Response) return admin;

const su = requireSuperAdmin(locals);         // 403 se não for Super Admin
if (su instanceof Response) return su;
```

⚠ **`requireAuth` sozinho não é autorização.** Ele prova que existe sessão, não que aquela sessão pode agir sobre aquele recurso. O capítulo 10 é inteiro sobre essa diferença.

8.3 Validação de body com Zod

```
import { validateBody } from '$lib/server/api';
import { prepararAssinaturaSchema } from '$lib/schemas';

const v = await validateBody(request, prepararAssinaturaSchema);
```

```
if (!v.ok) return v.response;      // 400 já formatado, com a primeira mensagem do Zod
const { signerName, latitude, longitude } = v.data;    // tipado
```

O helper cuida do JSON inválido (que estouraria uma exceção) e do erro de schema, devolvendo a `Response` pronta. O handler só precisa retorná-la.

8.4 Fetch no cliente — sempre por `$lib/api-fetch`

❌ Não escreva `fetch()` cru com `csrfHeaders()` e parse de erro à mão em componente novo.

✅ Use:

```
import { apiFetch, apiFetchResponse } from '$lib/api-fetch';

// API JSON: injeta CSRF, faz parse e lança Error com a mensagem do servidor
const dados = await apiFetch<{ ok: boolean }>('/api/gise/1/finalizar', { method: 'POST' });

// download/blob: mesmo tratamento de erro, devolve a Response crua
const res = await apiFetchResponse(`/api/escalas/${id}/download`);
const blob = await res.blob();
```

O que você ganha de graça:

- headers CSRF injetados;
- falha de rede vira "Erro de rede. Tente novamente." em vez de `Failed to fetch`;
- resposta de erro vira `Error` com a mensagem **do servidor** — e, quando há `errorId`, ele é concatenado (`"... (#3f2a91bc)"`). Sem isso o usuário lia "reporte o código de rastreamento" e não via código nenhum;
- `AbortError` é preservado, então buscas com `AbortSignal` continuam distinguindo cancelamento de erro real.

Para download de arquivo no navegador, existe outro par obrigatório:

```
import { baixarBlob, nomeArquivoContentDisposition } from '$lib/utils/download';
```

Nunca monte âncora + `createObjectURL` à mão — os call sites antigos esqueciam o `revokeObjectURL` e vazavam memória.

`fetch` cru só se justifica em **POST de form action do SvelteKit** (body `FormData`).

8.5 Cabeçalhos de resposta

Duas constantes/funções que você deve usar em vez de escrever à mão:

CACHE_PRIVADO (`'private, no-store'`) — para toda resposta autenticada que carrega dado pessoal: PDF assinado, manifesto forense, planilha de pessoal, termo de presença.

⚠ `no-cache` **não** serve, e essa é a armadilha: ele significa "revalide antes de usar" e **não proíbe** um cache compartilhado (edge da Cloudflare, proxy corporativo) de **armazenar** a resposta. Quem proíbe armazenamento é `no-store`. O valor estava escrito à mão em dezesseis lugares e as cópias divergiram exatamente como se esperava: a rota **pública** de download usava `private, no-store` com cinco linhas de comentário explicando o risco, enquanto as rotas **autenticadas** — servindo o mesmo blob, com o mesmo manifesto — usavam `no-cache`.

`contentDisposition(nome)` — monta o cabeçalho de download conforme a RFC 6266, com as duas metades (`filename=` e `filename*=`) escapadas segundo gramáticas diferentes. Isso importa porque **o nome pode vir de fora**: as rotas que servem a portaria anexada usam o nome escolhido por quem subiu o arquivo. Um nome terminado em `\` quebrava o parser e fazia o resto do cabeçalho ser lido como parte do nome.

9. Autenticação, sessão e onboarding

9.1 O fluxo de login por senha

1. O usuário informa **matrícula + senha**.
2. O servidor verifica com **PBKDF2-HMAC-SHA256**, 100 mil iterações (o teto do runtime da Cloudflare), salt de 16 bytes, comparação *timing-safe*. Em produção, a senha passa antes por um HMAC com o `PASSWORD_PEPPER` — é o formato `pbkdf2v3`.
3. **2FA obrigatório**: código de 6 dígitos enviado por e-mail. É *fail-closed* — conta sem e-mail cadastrado **não** recebe sessão.
4. Sessão criada com token de 256 bits.
5. Cookie `session_token` — `httpOnly`, `secure`, `SameSite=strict`.

O **TTL da sessão é de 1 hora** (`SESSION_TTL_MS` em `src/lib/auth.ts`), com *sliding*: cada uso estende o vencimento no banco, e o cookie é reemitido junto (§6.3). O plano de remediação LGPD pediu 1 h; o valor só pôde ser reduzido depois que o *sliding* foi consertado nas duas metades.

📖 O `README.md` ainda cita 8 horas nesta seção — o código é a fonte de verdade. Quando encontrar divergência assim, o certo é corrigir o documento vivo no mesmo PR em que você mexer na área.

9.2 Por que o pepper importa

O `PASSWORD_PEPPER` é um segredo **global**, aplicado por HMAC sobre a senha antes do PBKDF2. Com ele, um vazamento do banco sozinho não permite força bruta offline: o atacante precisaria também do segredo, que vive na configuração do Cloudflare e não no banco.

⚠ **Nunca rotacione o pepper sem plano de migração.** Trocar o valor invalida todos os hashes `v3` existentes — o que na prática significa reset de senha para todo mundo. O mesmo vale para `CPF_ENCRYPTION_KEY` e `CPF_INDEX_KEY`: são chaves *load-bearing*.

9.3 Login por certificado digital A3

Alternativa ao par senha + 2FA. O usuário assina um desafio com o e-CPF; o servidor verifica:

- a assinatura do desafio;
- a cadeia até uma raiz ICP-Brasil confiável;
- a **revogação** do certificado por OCSP — um e-CPF revogado é recusado. Se o respondedor da autoridade certificadora estiver fora do ar, o login prossegue e registra `metadados.ocsp = 'unknown'` na auditoria (*soft-fail*).

O botão existe nas duas abas do `/login`: na aba **Policial** cria sessão operacional; na aba **Administrador** resolve a conta admin vinculada ao policial do certificado.

9.4 Uma senha, duas identidades

Este é um conceito do domínio que confunde quem chega. O **Admin Geral vinculado** é uma pessoa com **duas linhas** no banco: uma em `policiais` e uma em `administradores`, esta com `policial_id` apontando para aquela. A linha de admin **não tem senha própria** — recebe um placeholder aleatório que ninguém lê. O login de administrador autentica contra `policiais.senha`.

Daí saem duas perguntas que todo fluxo de credencial precisa responder, e as duas têm resposta única em `src/lib/server/auth/credencial.ts`:

Pergunta	Resposta
Onde gravar a senha nova?	<code>resolverCredencial().dono</code> — a linha que o login lê
Quais sessões derrubar?	<code>revogarSessoesDaCredencial()</code> — as duas identidades

⚠ Gravar na linha errada não gera erro: a senha nova simplesmente não passa a valer, e a antiga continua valendo. Derrubar só o cookie do modo atual deixa vivo o outro, destravado pela mesma senha.

A **alternância de acesso** (botão na barra superior, `/api/auth/alternar-acesso`) troca entre modo Administrador e modo Usuário sem novo login. Ela **não concede privilégio novo** — só aponta a sessão para a outra identidade da mesma pessoa. Quem não tem conta vinculada não vê o botão, e o endpoint responde 403. Cada alternância é auditada.

9.5 Onboarding: duas fases, nesta ordem

1. **primeiro acesso** — o usuário define a própria senha e confirma o **e-mail pessoal** por código. O e-mail é o canal de recuperação: sem ele confirmado, uma senha perdida deixa a conta inacessível.
2. **Aceite do Termo de Uso vigente**.

O aceite é obrigatório a cada versão nova. Qualquer mudança em `src/lib/server/termo/termo-vigente.ts` gera um hash novo, que invalida os aceites anteriores e exige reaceite na próxima sessão.

9.6 Recuperação e limites

- Reset de senha por token enviado ao e-mail pessoal.
- `login_attempts` e `recovery_attempts` sustentam rate-limits.
- `RATE_LIMIT_IP_SALT` troca a chave do rate-limit de `"/24 anonimizada"` para hash salteado do IP completo — o que evita travar uma delegacia inteira, onde todos saem pelo mesmo endereço.

- Rotas que **montam documento** (PDF de escala, comprovante avançado) têm teto **por conta**, não por IP (`$lib/server/rate-limit-pesado`). São rotas autenticadas e autorizadas: o que faltava não era permissão, era custo. Contar por conta evita que um plantão em laço derrube o download dos colegas.

10. Autorização: a regra que define este projeto

Se você lembrar de uma única frase desta apostila, que seja esta:

Esconder o botão na tela não é autorização. O POST direto tem de morrer no servidor.

10.1 O que é uma "operação material"

Toda mutação de API (`POST`, `PUT`, `PATCH`, `DELETE`) e toda form action do SvelteKit é uma **operação material**: ela muda estado. E toda operação material precisa **recusar alguém** no servidor.

Isso é verificado no CI por `npm run guard:autorizacao`.

10.2 Por que não existe um `autorizar()` único

Seria a primeira ideia de qualquer um — e está registrado no `CLAUDE.md` por que não é assim: **a regra difere por domínio de verdade**.

Domínio	Como a permissão é decidida	Resolvedor
Escala ordinária	Lotação + escopo administrado + solicitação de assinatura	<code>verificarPermissaoEscala</code>
Escala extra (GISE)	Participação da seccional, quadro de supervisão ou vínculo de equipe	<code>verificarPermissaoGise</code> , <code>resolverParticipacaoGisePolicial</code>
Cadastro de policial	Escopo administrado pelo papel	<code>lotacoesAdministradas</code> , <code>ficha-permissao.ts</code>
Plano operacional	Portão próprio do módulo	<code>\$lib/server/planos/permissoao</code>
Perfis globais	Helpers de <code>\$lib/server/api</code>	<code>requireAdmin</code> , <code>requireSuperAdmin</code>

Uma lista fechada de nomes de helper nunca estaria completa — e deixaria passar justamente o handler novo com o resolvedor novo, que é o caso perigoso. Por isso o guard olha o **resultado**, que é fechado:

Nível	O que a operação faz	Como aparece no código
2	Recusa por permissão	<code>fail(403)</code> , <code>forbidden()</code> , <code>requireAdmin</code> , <code>requireSuperAdmin</code> , <code>exigirAdminGeral</code>
1	Só exige sessão	<code>fail(401)</code> , <code>unauthorized()</code> , <code>requireAuth</code>
0	Não recusa ninguém	—

Níveis 0 e 1 existem legitimamente: login não tem sessão para exigir, trocar a própria senha não tem segundo sujeito para autorizar, webhook se autentica por segredo compartilhado. Essas dispensas ficam declaradas **com o motivo** em `scripts/guard-autorizacao.mjs`.

💡 Declarar é o ponto. A diferença entre "público de propósito" e "esqueceram o guard" não está no código, só na cabeça de quem escreveu — e ali ela fica escrita. Encolher aquela lista é progresso.

10.3 A regra do recurso, não só do usuário

Uma armadilha sutil e frequente: verificar **quem** está chamando não basta se a operação lê um id vindo de fora da URL (corpo, `FormData`). Você precisa conferir que **aquele recurso pertence ao escopo de quem chamou**.

Foi assim que um membro de outra escala virava editável por id (achado `FLW-ESC-002`): o handler checava que o usuário era admin de alguma unidade, aceitava o `membro_id` do corpo e editava — sem conferir que aquele membro estava numa escala do escopo dele.

10.4 Os helpers obrigatórios

Para regressões já vistas, o guard exige o helper **no corpo do handler** — import no arquivo ou um 403 genérico não bastam. Exemplos:

- assinar escala precisa passar pelo portão `carregarEscalaParaAssinatura`, não por `podeAssinarEscala` remontado à mão;
- rotas de GISE precisam de `carregarGiseParaAssinatura`.

Essa exigência tem uma história bonita. O portão de assinatura de escala rodava copiado em cinco `+server.ts`, e uma das cinco não recusava escala do tipo FDS. Não havia buraco explorável na prática, mas era a quinta cópia esperando que alguém removesse a recusa do lugar que ainda a tinha.

O portão de GISE, extraído na mesma leva, mostrou algo mais interessante: as cinco rotas divergiam em **dois eixos independentes**, cada um numa cópia diferente. Uma não checava status; e o `preparar-assinatura` era a única que **não** admitia Admin Geral — o que tornava a permissão de admin no `finalizar` **inalcançável**, sintoma de que ela nunca deveria ter existido. A decisão final veio da interface: não existe caminho na tela para um Admin Geral assinar a escala GISE, então as quatro rotas que aceitavam `tipo === 'admin'` liberavam por POST direto exatamente o que a tela nunca ofereceu.

A lição registrada no `CLAUDE.md` não é "extraia e resolva na hora". É que **a extração torna a pergunta formulável** — enquanto eram cinco cópias, não havia o que comparar para notar a contradição.

10.5 A matriz de papéis

Tipo	Papel	Alcance
<code>admin + isSuperAdmin</code>	Super Admin	Tudo do Admin Geral, mais: promover admins, gerenciar policiais e unidades, configurar política de assinatura, consoles de auditoria, baixar o forense em <code>/validar</code>
<code>admin</code>	Admin Geral	Operação global (escalas, GISE, LGPD) em todas as unidades. Não remodela a base
<code>policial</code>	<code>admin_seccional</code>	Escalas da sua seccional; solicita correções cadastrais e atos de RH; informa linha de base; vê produtividade escopada
<code>policial</code>	<code>admin_unidade</code>	O mesmo, escopado à sua unidade
<code>policial</code>	—	Apenas as próprias escalas e participações

Além disso, contas de administrador têm **módulos** liberados por conta (`administradores.modulo_escalas`, `modulo_gise`, migração 0065). O cookie `admin_modulo` é só preferência de navegação; as flags é que são permissão, e `adminPodeAcessarRota` as aplica no hook.

10.6 O complemento executável

O guard lê o código. O spec `e2e/autorizacao-negativa.spec.ts` **executa**: ele varre `src/routes/` **em tempo de teste e exercita** todas** as operações materiais em dois cenários — anônimo e policial de outra unidade contra um recurso real — exigindo 401/403/404 e nenhum documento criado ou apagado.

A tabela não é escrita à mão: rota nova entra sozinha, sem depender de alguém lembrar.

⚠ Duas armadilhas ao mexer nele: alvo protegido por **outro** motivo (escala já assinada, GISE fechada) não testa permissão — o 409 chega primeiro e esconde a falta do 403; e form action **não** usa status HTTP, porque o `ActionResult` viaja em JSON sob 200.

Parte III — As regras inegociáveis

Os cinco capítulos desta parte descrevem as regras que o CI verifica e que nasceram de incidentes reais. Elas não são preferência de estilo: cada uma tem uma história de bug atrás.

11. Entrada validada no servidor

11.1 O problema

Autorizar diz **quem** pode agir. Falta dizer **o que** pode entrar — e as duas metades do projeto tratam isso de forma desigual:

- **Rota de API tem trilha.** O padrão obrigatório é `validateBody` com Zod, que traz `.max()`, faixa e enum de graça.
- **Form action lê `FormData` na mão**, e cada autor decidiu sozinho o que conferir. A maioria decidiu por *truthiness* (`if (campo)`), que aceita qualquer coisa não vazia.

Isso não é hipótese. **Toda** falha de entrada da auditoria de setembro de 2026 caiu do lado das form actions, sempre com a mesma forma: a regra certa num caminho, ausente no irmão.

Entrada aceita	Consequência
<code>mes=99</code>	<code>MESES_PT[98]</code> → <code>undefined</code> no título do documento, enquanto a rota <code>criar</code> ao lado passava por <code>escalaSchema</code>
Vagas com <code>999999</code>	Desarmava o <code>COUNT(*) < slots</code> que aloca membros
<code>observacoes</code> sem limite	Texto arbitrário entrando dentro de PDF assinado
<code>data_inicio='banana'</code>	Liberava presença fora do horário, porque <code>horarioGiseLiberado</code> falha aberto em data que não parseia

Repare no último: a função de horário não recusava a data inválida — ela devolvia "liberado". Uma validação ausente virou um portão desligado.

11.2 As cinco formas aceitas de limitar

Verificado no CI por `npm run guard:entrada`. O guard aceita:

1. **Schema Zod** — `validateBody(request, meuSchema)` nas rotas de API.
2. **Leitor de `$lib/server/form-data`** — nas form actions:

```
import { textoLimitado, inteiroNaFaixa, dataIso, horaHhMm, MAX_OBSERVACOES } from
'$lib/server/form-data';

const observacoes = textoLimitado(fd, 'observacoes', MAX_OBSERVACOES);
const mes = inteiroNaFaixa(fd, 'mes', 1, 12);
```

```
const inicio = dataIso(fd, 'data_inicio'); // null se não for YYYY-MM-DD válido
const hora = horaHhMm(fd, 'hora_inicio'); // null se não for HH:MM
```

1. **Leitura booleana** — `fd.get('ativo') === 'on'` não tem como extrapolar.
2. **Identificador** — que se valida por **posse** (é a pergunta do capítulo 10, não desta).
3. **Comparação escrita à mão** — `if (mes < 1 || mes > 12) return fail(400, ...)` conta.

Limite que vem de função de domínio (`erroDeDatasForaDoPeriodo`, `lotacaoNoEscopo`, `parseRespostasFormularioJsonStrict`) entra numa lista `LIMITADO_POR` **pelo nome**, e o `guard` confere que a chamada existe no corpo do handler — entrada que virou promessa vazia reprova em vez de dar verde.

11.3 O corolário prático

Campo com `maxlength` na tela precisa do mesmo número no servidor.

`maxlength`, botão escondido e `disabled` são **dicas de digitação**: somem num POST direto, num `curl` ou com uma linha no devtools.

Um exemplo real e instrutivo: as `observacoes` da escala tinham `maxlength=500` em quatro telas, servidor sem limite e coluna sem `CHECK`. Hoje o limite é uma constante (`MAX_OBSERVACOES`) lida pelas telas que editam e pelas actions que gravam.

⚠ Um detalhe que engana: regex de e-mail prova o **formato** e casa com string de qualquer tamanho. O limite de tamanho é outra coisa — `MAX_EMAIL` (254).

11.4 A baseline, e por que ela está vazia

O `guard` trabalha contra uma `BASELINE`: o que já existia está aceito, só o **novo** reprova. Hoje ela está **vazia** — as 27 leituras que existiam quando o `guard` nasceu foram pagas em duas levas.

Vazia ela continua útil: é o lugar declarado para a próxima dívida que alguém precise assumir conscientemente.

⚠ **Encher a baseline para o guard passar troca um achado por uma linha de código.** Se você chegou nesse ponto, pare e valide a entrada de verdade.

11.5 Um caso que vale estudar: a trava que não travava

O gate que exigia GPS na assinatura checava `typeof latitude !== 'number'`. O problema: `typeof NaN === 'number'`. O cliente manda a coordenada em texto, o servidor faz `parseFloat`, então `latitude=abc` chegava como `NaN`, passava no gate, e a assinatura seguia com o manifesto imprimindo "Não capturado".

Pior: `latitude=999` também passava — e essa o manifesto **imprime** como o local do ato. Evidência inventada corrói a credibilidade das outras evidências do mesmo documento.

Hoje o gate é `coordenadaGeograficaValida` (finito e dentro da faixa), e coordenada implausível é persistida como ausência mesmo com a flag desligada.

💡 Guarde a lição geral: **validar tipo não é validar valor**. NaN é number, Infinity é number, -1 é um inteiro perfeitamente válido para um campo que só admite positivos.

11.6 Ausência declarada

Há um caso em que o servidor **aceita** a evidência ausente: a presença GISE tem janela de horário e é base de pagamento de diária, então recusa seca deixaria de fora quem tem o GPS negado pelo aparelho.

A saída foi aceitar **ausência declarada**, com motivo vindo de lista fechada (`permissao_negada`, `indisponivel_no_aparelho`, `falha_tecnica`), registrado na trilha de auditoria.

Sendo preciso sobre o que isso compra: **não impede quem quer burlar** — um cliente adulterado sempre pode declarar "GPS negado". O que muda é que a ausência deixa de ser **invisível**: antes o servidor gravava sem GPS e sem registrar nada; agora todo ato sem evidência carrega o motivo, e o padrão fica contável no console de auditoria. Um servidor que declara "GPS negado" em todas as presenças aparece.

12. Duplicação: extrair antes de comentar

12.1 A regra

Achou a mesma lógica em dois lugares? Extraia — não comente as duas.

12.2 Por que comentar não resolve

Os bugs corrigidos em julho de 2026 têm todos a mesma forma: lógica copiada, uma cópia consertada, as outras não. E na maioria a cópia **correta** vinha acompanhada de um comentário explicando a armadilha — que não protegeu ninguém.

Bug	O que a duplicação escondia
<code>message.includes('UNIQUE')</code> em 4 lugares	Violação de unique virava 500 com SQL cru em vez de 409
<code>getField('serialNumber')</code>	CPF vazio no <code>/validar</code> para e-CPF sem <code>:CPF</code> no CN
Shades Tailwind inexistentes	Classes que não geravam CSS nenhum
Slot removido sem as equipes	Membros invisíveis na tela e ativos no gate de presença
<code>toISO</code> com duas convenções de mês	Data de um mês errado, sem erro nenhum
<code>hoje()</code> com <code>toISOString()</code> (2 lugares)	Calendário marcava amanhã das 21h à meia-noite
Laço "dias do intervalo" (3 lugares)	A mesma troca local ↔ UTC, latente em fuso positivo
"Restrito ao Admin Geral" (5 arquivos)	O gate era Super Admin; o comentário convidava a afrouxar
Portão de assinar escala (5 rotas)	Uma das cinco não recusava escala FDS
Fallback de hora do plantão (3 lugares)	<code>'08:00'</code> numa tela, <code>'08'</code> nas outras
Portão de assinar GISE (5 rotas)	Uma não checava status; quatro admitiam admin sem UI

Comentário protege quem lê aquele arquivo. Extração protege quem não sabe que o arquivo existe — que é justamente quem quebra o sistema.

E há um agravante: comentário **errado** sobre gate de permissão é pior que comentário nenhum, porque alguém vai "consertar a inconsistência" na direção da frase.

12.3 O guard

`npm run guard:duplicacao` reprova bloco de 10 linhas repetido entre arquivos, a menos que já esteja em `scripts/duplicacao-baseline.json`.

A baseline existe porque **não há meta de "0% duplicado"** — veja o corolário abaixo. As decisões de **manter** moram lá, cada uma com o motivo no campo `nota`.

 `--atualizar` regrava a baseline, e **não é o jeito de fazer o guard passar**: regravar sem extrair troca um achado por uma linha de JSON, que é a versão automatizada de "comentar em vez de extrair".

O guard tem um limite conhecido: bloco menor que 10 linhas relevantes é invisível para ele. Ele reduz a classe do problema, não a elimina.

12.4 O corolário: quando NÃO extrair

Se a extração exigir tantos props que o componente comum fique pior que a duplicação, **registre a decisão no código** em vez de extrair. Há dois exemplos no repositório: a grade dos três calendários e o barrel `lib/db.ts`.

Registrar significa escrever, no arquivo, por que as duas versões existem e o que deve mudar junto. Não é desculpa — é documentação de uma escolha.

13. LGPD, dado pessoal e minimização

13.1 O que é dado sensível aqui

O sistema guarda, de servidores públicos: nome, matrícula, **CPF**, telefone, e-mail, lotação, cargo, classe — e, nos atos de assinatura e presença: **selfie**, **coordenada GPS**, **endereço IP** e user-agent.

O conjunto CPF + IP + GPS + selfie é o que o projeto chama de **material forense**. Ele existe porque é o que dá força probatória ao documento; e justamente por isso o acesso a ele é restrito.

13.2 Cifra em repouso e índice cego

O CPF é gravado **cifrado** (AES-256-GCM, prefixo `enc:v1:`) quando `CPF_ENCRYPTION_KEY` está configurada.

Isso cria um problema clássico: como buscar por CPF sem decifrar a tabela inteira? A resposta é o **índice cego**: uma coluna `cpf_index` com o HMAC-SHA256 do CPF, usando uma chave **distinta** (`CPF_INDEX_KEY`). O lookup vira `WHERE cpf_index = ?`, determinístico, sem decifrar nada.

 As duas chaves são *load-bearing*: trocá-las invalida os dados existentes.

13.3 Minimização por projeção

A regra: **recorte antes de serializar, e de preferência no `SELECT`**.

Duas correções reais mostram por que isso importa mais no *payload* do que na tela:

- `/gise/[id]` e `/escalas/[id]` devolviam ao navegador o `assinante_cpf` **completo** (decifrado), enquanto a API do mesmo campo mascarava para todo mundo que não fosse Super Admin;
- `/gise/[id]` mandava a linha **crua** de `buscarPresencasGise` — CPF, IP, user-agent, latitude, longitude e as chaves R2 das selfies de cada integrante — para o Admin Geral, o admin de seccional e o **supervisor**, que é policial comum.

Nada disso aparecia na tela. Por isso a revisão visual não flagra: o excesso está no dado hidratado, não no que se vê.

As duas regras que saíram disso, cada uma com teste:

1. **CPF de assinante que sai do servidor passa por `cpfAssinanteParaExibir`**, que decifra e mascara na mesma chamada. Esse é o ponto: o helper conveniente já é o recortado, então não se obtém a versão crua por distração. `decifrarCpfDoDB` direto continua correto para quem **gera** o manifesto — ali o dado forense é o produto.
2. **Presenças que vão à tela passam por `presencasParaCliente`**, que devolve `policial_id` + entrada/saída e nada mais. É um objeto **novo, campo a campo** — não um `delete` das chaves indesejadas. A diferença importa: coluna nova em `buscarPresencasGise` (e evidência é o tipo de tabela que cresce) não passa a viajar de graça.

13.4 Quem alcança o forense

Só o **Super Admin** (`podeBaixarForense`, em `src/lib/server/assinatura/cpf-assinante.ts`). O portal `/validar` já diz isso por escrito ao visitante: "IP, user-agent e GPS: omitidos".

13.5 Retenção e direitos do titular

- `/api/admin/lgpd/incidentes` e `/api/admin/lgpd/solicitacoes` — console de conformidade (Admin Geral).
- `/api/lgpd/solicitar` — solicitação do titular (art. 18 da LGPD).
- `/api/webhook/limpeza-retencao` — a rotina de retenção, disparada por webhook autenticado (não há cron dentro do Worker). Ela purga também o `app_log`, com prazo configurável (`lgpd.retencao.app_log_dias`, padrão 90 dias).

13.6 A régua para código novo

Antes de devolver qualquer coisa de um `load` ou endpoint, pergunte:

1. **A tela usa este campo?** Se não, ele não deveria estar no `return`.
2. **Quem vai receber isso?** Um supervisor é policial comum; um admin de seccional não é Admin Geral.
3. **Existe um helper que já recorta?** Se existe, use-o; se não existe e o campo é sensível, crie-o — e faça o helper conveniente ser o recortado.

14. Assinatura digital e documentos com valor jurídico

Este é o capítulo mais denso, e o que mais diferencia este sistema de um CRUD. Leia-o antes de tocar em qualquer coisa que produza PDF.

14.1 As três modalidades

Modalidade	Mecanismo	Evidências coletadas
Qualificada	e-CPF ICP-Brasil via WebPKI (Lacuna) ou Assinador SERPRO Desktop	Certificado, OCSP, carimbo de tempo (CADES-LT)
Avançada	2FA por e-mail (sempre) + selfie com <i>liveness</i> + GPS + selo institucional	Foto, coordenadas, user-agent, timestamp
Simples	Confirmação textual — descontinuada , restrita a fluxos FDS legados	IP, user-agent, timestamp

O enquadramento jurídico (Lei 14.063/2020, MP 2.200-2) está num parecer arquivado no histórico do Git.

 Quem decide o nível é o campo `cms_sha256`, nunca a existência da linha. A tabela `gise_presenca_termos` recebe termo dos **dois** fluxos que produzem PDF de presença: Token A3 (qualificada) e passkey (avançada). Só o `cadetes-finalizer` grava `cms_sha256`, então é ele a régua. Classificar por "tem termo?" fazia o manifesto imprimir **QUALIFICADA · ICP-BRASIL** sobre uma presença por passkey — e afirmar ICP-Brasil onde não há certificado ICP derruba, em perícia, a credibilidade das **outras** evidências do mesmo documento.

14.2 O fluxo de dois tempos

A chave privada do policial está no token dele, não no servidor. Por isso a assinatura acontece em dois tempos:

```
preparar → (o dono da chave assina o digest) → finalizar
```

1. **Preparar** — o servidor gera o PDF, abre o placeholder PAdES, calcula o digest a assinar e devolve ao cliente.
2. O cliente aciona o WebPKI ou o SERPRO Desktop, que assina o digest com a chave do token.
3. **Finalizar** — o cliente devolve a assinatura; o servidor embute, valida e grava.

14.3 A intenção de assinatura

Entre os dois tempos, o PDF passa pelo **cliente**. Até agosto de 2026 o `finalizar` aceitava de volta qualquer `preparedPdf` e o gravava no recurso da URL — e a verificação criptográfica não acusava nada: a assinatura **era** válida e o CPF do certificado **era** o do usuário logado. Só o documento podia ser outro.

Três buracos, fechados pela tabela `assinatura_intencoes`:

Buraco	Como era explorável	Como foi fechado
Alvo	Preparar na escala A e finalizar na B guardava o PDF de A como documento de B	A intenção carrega <code>recurso + recurso_id (+ escopo_id)</code> , e o finalizar confere contra a URL
Uso único	A mesma preparação podia ser finalizada quantas vezes quisessem	Consumo por <code>UPDATE ... WHERE usado = 0 ... RETURNING</code> — o SQLite serializa, exatamente um vencedor altera a linha
Conteúdo	Assinar o digest de um PDF e enviar outro no corpo	O <code>preparedPdf</code> é conferido contra o <code>documento_hash</code> gravado pelo servidor

De quebra, o `verificacao_hash` deixou de vir do cliente — era ele que escolhia a chave no R2 e o código público do `/validar`.



A intenção não substitui a permissão. O finalizar continua chamando `verificarPermissaoEscala/verificarPermissaoGise`: a intenção prova que **este** ator preparou **este** documento para **este** alvo, não que ele **ainda pode** assiná-lo — permissão pode ter sido revogada no meio do caminho.

14.4 O ciclo comum e o portão

O miolo comum dos quatro `preparar-assinatura` vive em `src/lib/server/assinatura/preparar-ciclo.ts`. Repare no desenho:

- `prepararAssinaturaPorToken` é o **portão** exportado;
- `fecharPreparacaoAssinatura` **não** é exportado, justamente para que uma rota não possa remontar a sequência à mão e esquecer um dos dois passos (gravar a cópia de conferência e registrar a intenção);
- a rota diz **onde** o campo de assinatura cai e **qual** é o alvo; o resto não é decisão dela;
- o módulo **não gera** o PDF de conteúdo e **não autoriza** — o caller já recusou quem não podia.



Esse padrão — exportar o portão e esconder as peças — é o que impede a próxima cópia divergente. Vale copiar o desenho quando você extrair algo parecido.

14.5 Verificação e validação pública

`/validar/[hash]` é **pública e sem autenticação**: qualquer pessoa verifica um documento pelo código impresso no PDF. O que o servidor faz:

- confere o hash do arquivo;
- verifica a assinatura CAdES e a cadeia até uma raiz **ICP-Brasil** (trust store em `src/lib/server/assinatura/icp-brasil/`);
- consulta **OCSP** (revogação);
- confere o **carimbo de tempo** (TSA), classificando-o em `act_icp`, `tsa_externa` ou `servidor`.

Visitante **autenticado** vê também o recorte da chave de assinatura, para confrontar com a ficha do servidor sem abrir o banco. O anônimo não recebe esse recorte. O Super Admin alcança o material forense.

14.6 As flags de política

`/conf-ass` controla flags como `exigir_foto`, `exigir_gps`, `restringir_smartphone`, `exigir_passkey_assinatura`. Elas são cacheadas por 5 minutos em todos os PoPs, e o cache é invalidado quando um admin altera a configuração.

⚠ **Todas são aplicadas no servidor** — inclusive `restringir_smartphone`, que recusa a assinatura avançada por user-agent não-móvel em vez de só esconder o painel. Uma flag que só escondesse a tela seria decoração.

14.7 Golden antes de refatorar

PDF assinado, e-mail transacional e termo de presença são documentos, não saída de função.

O ritual obrigatório antes de tocar em qualquer um dos três:

1. rode o harness (`pdf-goldens`, `email-templates`) e confirme verde;
2. refatore;
3. confirme que **não mudou um byte**.

`UPDATE_PDF_GOLDENS=1` e `UPDATE_EMAIL_GOLDENS=1` regravam os goldens — use **só** quando a mudança visual for intencional, e confira o arquivo gerado antes de commitar.

⚠ Regravar para "fazer o teste passar" altera silenciosamente um documento que alguém já assinou.

15. Observabilidade e auditoria

15.1 Dois registros, dois propósitos

Console	Fonte	Conteúdo
<code>/auditoria</code>	<code>audit_log</code>	Trilha forense de eventos de negócio: catálogo de ações, ator × alvo, severidade, cadeia de hash verificável, exportação CSV/PDF
<code>/auditoria/logs</code>	<code>app_log</code>	Logs técnicos : todo <code>logger.warn</code> / <code>logger.error</code> do servidor

Os dois são restritos ao Super Admin.

15.2 O fio que liga tudo: `request_id`

O `requestId` é gerado no primeiro hook e propagado por `AsyncLocalStorage`. Ele é, ao mesmo tempo:

- o **errorId** que o usuário vê num erro 5xx;
- a **tag** enviada ao Sentry;
- a coluna que correlaciona `audit_log` e `app_log`.

No detalhe de um evento da auditoria, o Request ID é um link para os logs técnicos daquela mesma requisição. Na prática: o usuário reporta oito caracteres, e o operador chega ao stack trace em dois cliques.

15.3 Como registrar

Log técnico:

```
import { logger } from '$lib/server/logger';
logger.warn('[gise/presenca] GPS ausente', { policialId, motivo });
```

Níveis `debug` e `info` **não** são persistidos em `app_log` (só saem no log do Cloudflare). `warn` e `error` são persistidos por requisição, **depois** da resposta (`waitUntil`), sem custo no caminho crítico.

Evento de auditoria:

```
import { auditar, contextoDeEvento } from '$lib/db';
await auditar(db, { acao: 'escala_assinada', ...contextoDeEvento(locals) , alvo_id:
escala.id });
```

A trilha tem **cadeia de hash**: cada evento carrega o hash do anterior, o que torna adulteração detectável. Evento que a cadeia recusa vai para `audit_pendencias` e é reprocessado.

15.4 O que auditar

Regra prática: **todo ato que a corporação precisaria explicar depois**. Assinatura, revogação de documento, aprovação de ato de RH, alternância de acesso, alteração de política de assinatura, exclusão de operação, login por bootstrap.

O catálogo de ações (`CATALOGO_ACOES`) descreve cada uma com severidade. Ação nova entra no catálogo — evento sem descrição vira linha ilegível no console de quem investiga.

Parte IV — Os módulos, por dentro

Esta parte é de **consulta**. Leia o capítulo do módulo em que a sua tarefa cai; os outros podem esperar.

16. Escalas ordinárias

16.1 O que o módulo faz

Gerencia o ciclo de vida das escalas de **plantão**, **expediente** e **fim de semana (FDS)**:

- criação e edição, com seleção de policiais por unidade;
- geração do PDF com layout oficial;
- envio por e-mail ao destinatário configurado;
- assinatura digital (e-CPF via WebPKI/SERPRO, ou avançada);
- validação pública por QR Code / hash.

16.2 Onde está

Camada	Caminho
Listagem	<code>src/routes/escalas/</code>
Detalhe/edição	<code>src/routes/escalas/[id]/</code>
Criação	<code>src/routes/escalas/nova/</code>
Caixa de entrada	<code>src/routes/recebidos/</code>
API	<code>src/routes/api/escalas/[id]/...</code>
Regras	<code>src/lib/server/escalas/</code>
Dados	<code>src/lib/db/escalas.ts</code> , <code>documentos.ts</code>

16.3 O modelo

Tabela	Papel
<code>escalas</code>	A escala mensal: lotação, tipo, mês/ano, período, observações
<code>escala_policiais</code>	Quem está escalado, em qual data, horário e equipe
<code>escala_documentos</code>	O PDF assinado, com metadados CAdES-LT (OCSP, carimbo, selfie, GPS, IP)
<code>escala_solicitacoes_assinatura</code>	Pedido de assinatura direcionado a uma unidade ou respondência

16.4 As regras que moram em `server/escalas/`

`conflict.ts` — **choque de horário**. Verificação **global**: impede o mesmo policial ser escalado no mesmo dia e horário em dois lugares, cruzando plantão, expediente, FDS e GISE. É o tipo de regra que precisa olhar fora do próprio módulo — um policial escalado numa GISE não pode estar num plantão ao mesmo tempo.

periodo.ts — **o dia pertence ao período?** Módulo puro, com uma história instrutiva (achado `FLW-ESC-005`). As datas chegam do cliente: `data_plantao` num campo, `datas` num JSON oculto montado pelo calendário. Nada no servidor conferia se caíam dentro de `[data_inicio, data_fim]` — e o calendário, única coisa que limitava, é **markup**.

O efeito de um POST direto não era um erro visível: a linha entrava, sumia da grade (que só desenha os dias do mês) e **reaparecia no PDF**, que lista o que o banco tem. Uma escala de setembro assinada com um plantão de agosto.

O módulo é separado e puro porque **quatro** actions precisam da mesma resposta: `adicionar`, `adicionarPlantao`, `repetir` e `editar`.

exclusao.ts — **caminho único de exclusão**. A ordem é obrigatória: objetos no R2 (blob assinado, cópia de conferência, selfie) → linha do documento → a escala. Se a escala fosse apagada primeiro, o cascade da FK apagaria o `r2_key` antes da limpeza e os objetos ficariam **órfãos e irrastráveis** no bucket.

Antes, três call sites (`/escalas`, `/recebidos`, `/painel`) repetiam a sequência à mão — e o do `/painel` divergiu, excluindo sem limpar o R2.

projetar-mes.ts — **projeção para o mês seguinte**. Expediente deduplica e lança no dia 1; plantão identifica a rotação e projeta o ciclo. As actions continuam donas de criar a escala, auditar e recusar conflito; daqui saem só as linhas e a lista de quem não pôde ser projetado.

permissao.ts — **quem pode ler, quem pode assinar**. São dois portões distintos:

Portão	Pergunta
<code>verificarPermissaoEscala</code>	Pode ler esta escala?
<code>podeAssinarEscala</code>	Pode assinar ?
<code>carregarEscalaParaAssinatura</code>	O portão completo das rotas de assinatura (é este que as rotas devem chamar)

A ordem de `verificarPermissaoEscala`:

1. Admin Geral → sempre.
2. Mesma lotação → sempre.
3. Admin de seccional/unidade cujo escopo cobre a lotação → direto.
4. Admin DPC fora desse escopo → só se houver **solicitação de assinatura** direcionada a ele.
5. Demais → negado.

sync-estado.ts — **carimbos de revalidação**. Um carimbo é uma string que muda quando o conjunto de dados da tela muda, e só então. O cliente faz *poll* em `GET /api/sync/estado` e refaz o `load` só quando o carimbo mudou. Por isso os carimbos são agregados (`count`, `max`) e não listagens: montar o carimbo tem de ser mais barato que montar o payload.

16.5 A assinatura da escala

Dois caminhos, com miolo comum em `assinatura-escala.ts`:

- **um tiro** (`assinar-simples`);
- **duas fases** (`passkey`) — existe porque a asserção `WebAuthn` precisa cobrir o hash do documento: o `preparar` monta o PDF e grava a intenção, a cerimônia biométrica acontece sobre esse hash, o `finalizar` sela e grava.

A ordem das operações não é livre e está fixada no módulo. Quando você mexer aqui, siga o ritual dos goldens (§14.7).

16.6 O que costuma quebrar

Sintoma	Provável causa
Data errada no PDF	Fuso: <code>toISOString()</code> em vez de <code>\$lib/utils/datas</code>
Policial escalado duas vezes	Alteração que passou por fora de <code>conflict.ts</code>
Objeto órfão no R2	Exclusão que não passou por <code>exclusao.ts</code>
403 ao assinar	Escala FDS, escala já assinada, ou permissão fora do portão

17. Escala extra: operações e GISE

17.1 O vocabulário, primeiro

GISE foi a primeira operação extraordinária do sistema, e o nome ficou colado em rotas e tabelas (`/gise`, `gise_escalas`). Hoje a aba se chama **Escala extra** e lista as escalas de **todas** as operações.

- **Operação** (`/gise/operacoes`) é o **catálogo**: nome, sigla, ciclo, quais tipos de equipe usa e a configuração de escala. Cada operação é dona dos **seus** formulários de produtividade.
- **Escala extra** é uma edição concreta daquela operação, com seccionais, equipes e membros.

 `/gise` é o prefixo **legado**. Telas novas do domínio não devem ser aninhadas sob ele (foi por isso que `/dados-base` nasceu na raiz). Se um dia a coerência de prefixo for perseguida, o caminho é renomear o módulo inteiro, não estender o nome antigo.

17.2 O modelo

Tabela	Papel
operacoes	Catálogo: tipos de equipe, ciclo, ativo , config de escala
operacao_linha_base	Valor inicial de cada indicador por (operação, unidade)
gise_escalas	A escala extra: status, supervisor, assessor, configuração
gise_seccionais	Seccionais participantes
gise_equipes	Equipes (operacional/SEINT) com slots DPC/OIP
gise_membros	Policial ↔ equipe
gise_presencas	Entrada/saída, com GPS e selfie
gise_presenca_termos	Termos de presença gerados
gise_documentos	PDFs assinados
gise_modelo_formulario	Modelo do formulário em JSON, por (operação, tipo de equipe)
gise_respostas_formulario	Respostas, em JSON
gise_assinaturas_relatorios	Assinaturas dos relatórios por seccional

17.3 Regras de exclusão e desativação

Operação com escala não se exclui — desativa-se. Escala histórica e PDF assinado continuam apontando para ela, e apagá-la deixaria documento entregue sem origem.

Operação que **nunca** recebeu escala — a criada por engano, a de teste — não é história de nada e ganha **Excluir**, levando junto os formulários dela.

⚠ Quem recusa é a **action**, recontando as escalas no servidor: a contagem que a tela mostrou pode ter envelhecido, e o botão escondido nunca foi autorização. Esse é o padrão a copiar quando você criar qualquer exclusão condicional.

17.4 Configuração em cascata

A configuração cai do mais específico para o mais geral:

colunas de gise_escalas → colunas de operacoes → configuracoes → constante do código

Campo vazio herda o padrão. E cuidado: **zero não é vazio** — 0 significa "esta equipe não tem essa vaga".

O horário segue a mesma ideia (gise_escalas → gise_seccionais → gise_equipes), com as duas últimas colunas **nuláveis**: nulo já quer dizer "o mesmo de cima". Na tela, horário **herdado** vira um ícone de relógio; horário **próprio** aparece escrito.

⚠ "Próprio" é comparação de **valor**, não de preenchimento. Quem salvava `08:00` numa equipe cuja seccional já era `08:00` ganhava um selo de "personalizado" que não personalizava nada. Quem responde é `horarioEfetivo / temHorarioProprio ( $lib/gise/horarios )`, que normaliza antes — `'08'` e `'08:00'` são a mesma hora, e essa diferença exata já produziu bug aqui.

17.5 O fluxo do membro

A visão do policial vive em `/res-gise`, em duas abas da barra lateral:

- **Presença GISE** — só aparece com escala ativa: confirmar entrada → entregar relatório → confirmar saída;
- **Histórico GISE** — participações encerradas (`?status=finalizadas`).

A presença registra **GPS e selfie**. Em desktop, a confirmação é por **Token A3**. O comprovante é baixável nos dois fluxos: Token A3 serve o termo qualificado do R2; presença em tela gera o comprovante avançado sob demanda.

Um padrão de UI que vale conhecer: tarefa longa vira modal, formulário longo vira rota. A página mostra o **estado** (barra de progresso + um quadro por passo) e cada passo abre um modal com o seu formulário — exceto o relatório de produtividade, que tem 19 perguntas de nível 0 mais filhos condicionais e por isso virou **rota própria com wizard** (`res-gise/relatorio/[giseId]`). Rota, e não modal, porque o preenchimento tem endereço, sobrevive a um reload e admite rascunho.

17.6 Rascunho local

Formulário longo grava o blob no `localStorage` a cada pausa de digitação (debounce de 800 ms), com chave por (registro, dono). A regra de restauração é explícita:

- aplique o rascunho sozinho **só quando não há nada no servidor**;
- havendo, o servidor manda e o rascunho vira uma **oferta** com botão.

⚠ **Nunca decida por comparação de relógios** — os carimbos do banco são hora local em texto e o do navegador é do aparelho. E limpe o rascunho ao entregar e trave o autosave nesse instante, senão um timer pendente regravava o que acabou de ser apagado.

17.7 Presença: o portão

`presenca-gate.ts` decide se a presença pode ser registrada. Duas coisas que já falharam e hoje estão fechadas:

- `horarioGiseLiberado` falhava **aberto** em data que não parseia (§11.1);
- as evidências (`exigirFoto`, `exigirGps`, ambas com padrão `true`) viviam apenas no `SignaturePad`, e um POST direto registrava presença sem nenhuma das duas — enquanto o painel do admin as anunciava obrigatórias. Hoje o gate é `$lib/assinatura-evidencia`, *client-safe* para a tela pedir pela mesma regra.

18. Produtividade

18.1 A ideia

Cada operação define um **formulário** (por tipo de equipe). Os membros respondem. O painel `/produtividade` consolida as respostas em indicadores, metas e gráficos, com filtro por operação.

18.2 O modelo do formulário

O modelo vive em `gise_modelo_formulario` como **JSON**. A tabela de tipos de pergunta é `$lib/gise/tipos-pergunta.ts`, e ela responde: quais tipos abrem listagem, quais aceitam sub-pergunta e **onde cada um grava no blob**.

⚠ **Mexer em tipo de pergunta começa por lá, nunca pelos componentes.** `chavesLista` é a fonte **única** da chave da resposta — é ela que o indicador de meta, o gráfico e a expansão do relatório consultam. Uma tabela paralela escrita à mão já custou aos tipos de lista o direito de virar gráfico, e ninguém notou porque o indicador continuava funcionando.

⚠ Ao acrescentar um tipo, a **expansão** em `db/gise/respostas.ts` é obrigatória no mesmo passo: sem ela o policial preenche, o dado é gravado e **some do PDF assinado sem erro nenhum**.

Três tipos estão **aposentados** (`mandados_maiores`, `prisoas_maiores`, `apreensoes_menores`): fazem o mesmo que `lista_detalhada`, mas gravando em chave fixa, o que os limita a uma ocorrência por formulário. Eles não foram removidos porque **trocar o tipo de uma pergunta troca a chave da resposta** — na prática, apaga o histórico dela. O editor só os oferece na pergunta que já está com um deles.

18.3 Indicadores e metas

Uma pergunta contável pode ser marcada como **indicador**. São três tipos de meta, e é o `metaTipo` que discrimina a união `IndicadorConfig`:

<code>metaTipo</code>	O que mede	Objetivo	Linha de base
<code>percentual</code>	Variação sobre o valor inicial (ex.: -20%)	aumentar / diminuir	exige
<code>absoluto</code>	Alvo fixo (mínimo por unidade)	aumentar / diminuir	não usa
<code>proporcao</code>	Cobertura: % do total atendido	não tem	não usa

A meta percentual exige a **linha de base** — o valor de partida da unidade —, informada em `/dados-base/[operacaoId]` pelo admin de unidade/seccional.

`proporcao` existe porque "atender 100% das ocorrências" não se mede com um número solto: 12 atendimentos são ótimos se houve 12 ocorrências e ruins se houve 40. Ela grava dois números (`${key}__total` e `${key}__parte`) e só o tipo de campo homônimo a aceita.

💡 A operação vai no **caminho** (`/dados-base/[operacaoId]`), não num seletor ao lado dos campos. O valor digitado ali é o denominador de um percentual divulgado: gravá-lo sob a operação errada muda o atingimento de uma unidade sem tocar em relatório nenhum. Com a operação no caminho, não há controle a errar.

18.4 O que entra no painel

Cada seção entra por um critério **diferente** — confundi-los foi a origem dos bugs de agosto de 2026:

Seção	Entra quando...
Indicadores e metas	a pergunta tem <code>indicador</code>
Colunas por unidade	a pergunta tem <code>grafico.colunas</code>
Ranking de unidades	a pergunta tem <code>grafico.ranking</code>
Detalhamento por tipo	a pergunta tem <code>grafico.detalhe</code> e comporta a quebra
Prisões (bloco fixo)	o formulário tem <code>prisoas_maiores</code> ou <code>mandados_maiores</code>

A marca é uma escolha, não uma consequência do tipo. Antes, toda pergunta contável virava card sozinha, e a quilometragem inicial da viatura ocupava espaço ao lado das prisões — sem jeito de tirá-la a não ser apagando o campo, o que apagaria a coleta.

O título do card não é o enunciado da pergunta. São públicos diferentes: a pergunta é escrita para quem **preenche** ("10. HOUVE APREENSÃO DE DROGAS?"), o card é lido por quem **acompanha**. O campo "Título no painel" grava em `rotulo_painel`, e `tituloNoPainel` resolve em três degraus: rótulo próprio → identidade do tipo (Drogas, Armas) → texto da pergunta.

⚠ Cor e unidade **não** cedem ao campo: 'g' diz onde o número foi somado, e trocá-la por digitação faria o card mostrar gramas chamando-as de outra coisa. E nada disso entra no relatório assinado — o PDF continua saindo com o enunciado que a pessoa respondeu.

18.5 Como cada resposta é somada

Decisão de `valorDaResposta` (`$lib/produtividade/apresentacao`), válida para as três formas: `sim_nao` conta ocorrências, droga soma peso normalizado em gramas (exibido em kg), tipos de lista somam a chave de quantidade. Eram três cópias dessa regra, cada uma com a sua tabela de chaves.

O gate do "Sim". Nos tipos que perguntam "houve X? → se sim, quantos", a quantidade só conta com a resposta em 'Sim'. O blob guarda o que foi digitado: quem preenche a listagem, muda de ideia e responde "Não" deixa o número lá. O relatório assinado sempre ignorou esse resto; o painel não ignorava — e contava produção que o PDF não mostra.

18.6 Eixo, ordem e exportação

"Visualizar por" é um eixo, não um filtro. A mesma resposta pertence às duas chaves (`seccional_id` e `unidade_id`); trocar de modo não recorta nada, só muda por qual delas a lista é somada. Por isso o total do painel não muda ao alternar.

Quem responde por agrupar, ordenar e recortar é `$lib/produtividade/agrupamento` — fonte única dos três consumidores. Três decisões dele:

- equipe sem slot de delegacia resolve para a própria seccional (senão a soma das linhas ficaria menor que o total, sem nada explicando);
- a ordem é semântica ("melhores"/"piores", não "maior"/"menor"); num indicador de redução, ordenar pelo número cru poria a pior unidade no topo de "melhores";
- valor **não avaliável** (`null`) vai sempre para o fim, nos dois sentidos: não é a pior, é a que não se sabe.

A **ordem dos cards** é dado próprio da operação (`gise_modelo_formulario.painel_ordem`), montada no botão "Organizar painel". Antes a ordem era a do modelo, e mover um card exigia mover a **pergunta** — o que renumera o enunciado e reordena o formulário que o policial preenche: arrumar a leitura mexia na coleta.

Exportação: "Baixar (imagem)" desenha cada card num canvas próprio e salva PNG; "Baixar (PDF)" é `window.print()`, e quem pagina é o navegador segundo o `@media print` da rota. Duas consequências: a quebra é controlada por CSS (`break-inside: avoid`), e os seletores precisam ser `:global()` porque todo card vem de um componente filho — escrito sem isso, o CSS compilava e **nunca valia**.

19. Plano operacional e custeio

19.1 O que difere da escala extra

	Escala extra	Plano operacional
Deslocamento	Não sai da circunscrição	Equipes se deslocam para outra cidade
Pagamento	Sempre hora extra	Diária ou hora extra, conforme regra
Modelo	Catálogo (<code>operacoes</code>) + escalas	Evento único, com equipes próprias
Rota	<code>/gise</code>	<code>/gise/planos</code>

Os dois nascem no mesmo botão de `/gise/operacoes`, que pergunta qual dos dois se está cadastrando — mas não compartilham tabela, rota nem PDF.

19.2 A regra de custeio

A diária é verificada primeiro; não sendo devida, o relógio decide. As duas verbas **nunca se somam**.

⚠ Essa recusa é política do DPI SUL, **não** do decreto: o Decreto nº 36.182/2024 permite percebê-las concomitantemente. Precisa estar escrito, senão quem ler o decreto depois "corrige" a inconsistência somando as duas.

Pergunta, nesta ordem	Rubrica
Deslocamento $\geq$ limite (padrão 100 km) e operação $\geq$ 4h, com parecer favorável	diária, no mínimo 1,5
Distância bastaria, mas a operação tem menos de 4h	vale o relógio
Distância bastaria, mas a equipe não tem hora de término	vale o relógio, e a tela avisa
Abaixo do limite: 08:00–18:00 em dia útil	sem custo
Abaixo do limite: 06:00–08:00 e 18:00–00:00 útil	hora extra normal
00:00–06:00 útil, ou fim de semana e feriado em qualquer hora	hora extra plus (+30%)

A ordem **não é comutativa**. Uma equipe que sai às 04:00 e percorre mais de 100 km: o relógio sozinho diria hora extra pelas horas de madrugada; a regra completa diz **diária**. A mesma equipe numa terça das 09:00 às 17:00 recebe **sem custo**, não diária — missão inteiramente dentro do expediente não extrapola a jornada.

19.3 De onde vêm os 100 km

O limite **não está no decreto**. Os números de lá são outros: 8 horas de jornada e 120 km (este só dentro de região metropolitana e combinado com ausência de extrapolação). Os 100 km são a aritmética do próprio decreto com a permanência **real**:

- o cálculo estimado do decreto é $2 \times \text{tempo de ida} + 3\text{h de permanência} > 8\text{h}$;
- a operação do DPI SUL dura **4h**, não 3h;
- com o número real o teste vira $\text{ida} > 2\text{h}$, que nas estradas do Ceará cai perto de 100 km.

Medido em 4.005 pares, $\text{km} \geq 100$ e $2 \times \text{ida} + 4\text{h} > 8\text{h}$ concordam em **96,5%**.

O limite é campo do Super Admin (`/config-custos`) e **congela na versão**: o plano guarda `custo_parametro_id`, então subir o limite amanhã não muda a rubrica de um plano de março já impresso. As 4 horas seguem constante nomeada (`DURACAO_MINIMA_DIARIA_HORAS`).

19.4 O parecer do decreto

Quem decide se a diária é devida é `$lib/diarias/` — domínio próprio, porque a futura aba de solicitação avulsa consome o mesmo motor:

Módulo	Pergunta
<code>contagem.ts</code>	Quantas diárias, e em qual mês cada uma cai
<code>jornada.ts</code>	A missão de um dia extrapolou as 8 horas?
<code>vedacoes.ts</code>	O que precisa de conferência humana
<code>parecer.ts</code>	O veredito, com o dispositivo citado

A quantidade é $N - 0,5$ com pernoite, escrita como **atribuição por dia** — cada dia vale 2 meias, o último vale 1. Somado dá o mesmo $2N - 1$, mas quebrado por mês responde ao teto de 15 diárias/mês do art. 13 sem uma segunda regra. **Saída às 23h da véspera é pernoite.**

! Os alertas não bloqueiam. A vedação do art. 4º, §1º, II exige três condições **juntas**, e a terceira é de relógio. Barrar pela geografia sozinha recusaria o que a lei permite — então o código alerta e uma pessoa decide.

19.5 A distância é medida

O deslocamento é `origem → briefing → destino`, não a reta entre as pontas. Jucás → Iguatu → Juazeiro do Norte dá **189 km** contra 114 km diretos.

O número vem de uma **matriz rodoviária gravada** (`distancias_municipios`, 16.836 pares), gerada uma vez por `scripts/gerar-distancias.mjs`. Três razões para tabela em vez de API na hora: **reprodutibilidade, disponibilidade e auditabilidade**.

Linha reta foi descartada com medição: em 12 pares reais o fator rodovia/reta variou de **1,10 a 1,62**, e em 3 dos 12 ela daria a rubrica errada — sempre para menos.

As fontes: **IBGE** para nomes e códigos, **Wikidata** (CC0) para a coordenada da sede, **OSRM/OpenStreetMap** (ODbL) para a rodovia — cuja licença permite **armazenar** o resultado, o que o Google Distance Matrix proíbe.

⚠ O campo continua editável (pode haver desvio real), mas **quem decide se o valor gravado é a medida ou uma correção é o servidor**, que remede na gravação e registra na auditoria. Confiar num campo do formulário deixaria o corpo do POST afirmar a procedência de um número que vira dinheiro.

NULL é "ninguém mediu ainda" e é distinto de zero — zero afirma que origem e destino são a mesma cidade.

19.6 Congelamentos

Três coisas congelam, e é importante entender o padrão:

O que congela	Onde	Por quê
Valores de hora extra e diária	<code>custo_parametros</code> (append-only)	Reemitir em junho o PDF de março deve devolver os mesmos números
cargo e classe do membro	<code>plano_equipe_membros</code>	São base de cálculo; não acompanham promoção posterior
Nome do signatário	<code>planos_operacionais.diretor_nome</code>	Sai impresso sob a assinatura

Nome, matrícula, lotação e telefone continuam vindo **vivos** do cadastro — são identificação, não mudam o valor pago.

19.7 As listas do plano

Briefing, cidade de origem e cidade de destino são **listas do plano** (`plano_opcoes`), não campo livre por equipe: com oito equipes saindo para três cidades, bastava um acento diferente para o anexo listar dois destinos onde só há um.

Duas mecânicas de banco valem estudo:

- `uq_plano_opcoes_padrao` é índice único **parcial** sobre `(plano_id, tipo)` WHERE `padrao = 1`. Trocar a padrão é limpar a anterior e marcar a nova no mesmo `batch` — duas abas não conseguem deixar duas padrões;
- `uq_plano_opcoes_valor` recusa valor repetido pelo índice, não por um `SELECT` antes.

A padrão é **copiada** para a equipe nova, não herdada por referência: a equipe guarda o **texto**. Isso permite remover uma opção sem esvaziar o destino de uma equipe montada. E o seletor da equipe oferece o valor próprio dela mesmo quando ele saiu da lista — um `<select>` cujo `value` não casa com nenhuma `<option>` exibe o primeiro item, e salvar sem tocar no campo trocaria o destino impresso.

19.8 O PDF e as pendências

GET `/api/planos/[id]/download` sai em três páginas: corpo com as nove seções numeradas e a assinatura do Diretor, **Anexo I** (uma tabela por equipe: efetivo, jornada, rubrica e valor por servidor) e **Anexo II** (consolidado por categoria).

CPF não entra no documento — minimização: o papel circula, e a classe basta para justificar o valor da linha.

⚠ **Classe vazia não vira R\$ 0 em silêncio.** Sem faixa resolvida, a linha sai como **pendência** e o endpoint de download **recusa com 409**, nomeando quem falta. Equipe sem custo com membro sem classe gera **aviso**, não pendência — ela pode virar com custo, e o problema tem de aparecer enquanto ainda há tempo.

O que a tela mostra e o que o PDF imprime saem da **mesma** chamada (`montarCustoDoPlano`), não de dois cálculos parecidos.

20. Cadastro: servidores, solicitações e unidades

20.1 A ficha com dois poderes

`/policiais/[id]` é uma tela só, com dois modos, e o portão que os separa é `ficha-permissao.ts`:

Modo	Quem	O que acontece ao submeter
<code>direto</code>	Admin Geral	Grava o cadastro / executa o ato na hora
<code>solicitacao</code>	<code>admin_seccional</code> / <code>admin_unidade</code>	Vira pedido pendente para o Admin Geral decidir

20.2 As regras do cadastro, e o porquê de cada uma

- **O servidor não pede alteração do próprio cadastro.** `/perfil` é leitura, mais as duas coisas que pertencem ao titular: o **e-mail pessoal** (canal de recuperação — trocá-lo exige a senha dele **mais** um código no novo endereço) e a **chave de assinatura**.
- **E-mail pessoal não é solicitável por administrador nenhum**, pelo mesmo motivo. Todo o resto é (`CAMPOS_SOLICITAVEIS` em `$lib/cadastro-campos.ts`).
- **Todo pedido vai com justificativa** de até 300 caracteres, que acompanha cada linha da fila — decidir sem o motivo à vista é decidir no escuro.
- **Lotação não é campo solicitável**: transferir servidor é **movimentação**, com data, NUP e portaria anexa. Dois caminhos abertos produziriam transferência sem portaria, indistinguível de uma com portaria depois de gravada.
- **O CPF não é decifrado para quem só pede.** O campo aparece em branco no modo `solicitacao` e serve para informar o número **novo** — ler o atual nunca foi necessário para isso.
- **Papel administrativo e Admin Geral aparecem, informativos.** Mostrar o estado responde "por que esta pessoa administra a minha unidade?" sem obrigar ninguém a perguntar; as actions que os alteram seguem exigindo `isAdminGeral`.

20.3 Um padrão para copiar: o efeito mora fora da action

O efeito dos três atos de RH (movimentar, afastar, desvincular) mora em `acoes-rh.ts`, e é o **mesmo objeto** nos dois caminhos: a ficha o executa direto, e a aprovação em `/solicitacoes` o executa a partir da linha do pedido.

⚠ Enquanto o efeito morava dentro das form actions, aprovar teria de **reescrevê-lo** — a forma exata dos bugs de cópia divergente do capítulo 12.

O PDF da portaria sobe **no momento do pedido**: é o que permite ao Admin Geral baixar a portaria **antes** de aprovar. Aprovado, a chave passa a pertencer ao evento em `policial_historico`; recusado, o objeto é apagado do bucket — nenhuma outra linha voltaria a referenciá-lo.

20.4 Unidades

`/unidades` é restrita ao **Super Admin**, não ao Admin Geral: o nome da unidade é a chave que amarra lotação, escala e cabeçalho dos documentos (§7.5).

- `listarUnidades` devolve só as ativas (usado por todo o sistema);
- `listarTodasUnidades` inclui as desativadas (usado só pela tela que as gerencia);
- não existe exclusão (§7.6).

20.5 Sincronização com a planilha

O cadastro pode ser alimentado por uma planilha Google via `scripts/GoogleAppsScript_Sync.gs`, que chama `/api/webhook/sync-policiais` e `/api/webhook/sync-unidades` autenticado por `SYNC_TOKEN`, com proteção contra *replay* (timestamp + nonce).

Existe ainda `/api/webhook/reset-policiais`, **destrutivo**, exigindo quatro camadas simultâneas — o capítulo 24 detalha.

21. Consoles administrativos

Rota	Quem	O que faz
<code>/painel</code>	Admin	Visão gerencial das escalas
<code>/recebidos</code>	Admin	Caixa de entrada de escalas recebidas para assinatura
<code>/solicitacoes</code>	Admin Geral	Fila de decisão: correções de cadastro e atos de RH
<code>/auditoria</code>	Super Admin	Trilha forense, com cadeia de hash e exportação
<code>/auditoria/logs</code>	Super Admin	Logs técnicos, correlacionados por <code>request_id</code>
<code>/conf-ass</code>	Super Admin	Política de assinatura (flags)
<code>/config-geral</code>	Admin Geral	Configurações gerais (provedor de e-mail)
<code>/config-custos</code>	Super Admin	Valores de hora extra e diária, versionados
<code>/super-admin</code>	Super Admin	Console de boas-vindas
<code>/termo</code> , <code>/termo/dpo</code>	Público	Consulta do termo de uso vigente e contato do encarregado
<code>/validar</code>	Público	Validação de documento assinado

💡 A sidebar tem **dois níveis**: tudo que é de escala extra vive sob o item "Escala extra", que ao ser clicado **substitui** o conteúdo da barra pelo submenu (não expande — cinco itens indentados devolveriam a lista comprida que o agrupamento veio desfazer). O nível é decidido ao **abrir**, pela rota. Quem vê cada item é `menu-visibilidade.ts`; e agrupar é apresentação — o recorte de verdade segue no servidor.

Parte V — Trabalhando no projeto

22. Padrões de código

22.1 Svelte 5 — o resumo executável

❌ Proibido em código novo	✅ Use
<code>export let x</code>	<code>let { x } = \$props()</code>
<code>let x = ''</code> com expectativa de reatividade	<code>let x = \$state('')</code>
<code>\$: y = x * 2</code>	<code>let y = \$derived(x * 2)</code>
<code>writable()</code> / <code>readable()</code>	<code>\$state</code> em arquivo <code>.svelte.ts</code>
<code>onMount()</code> para lógica reativa	<code>\$effect()</code>
<code><slot /></code>	<code>{#snippet}</code> / <code>{@render}</code>
prop com two-way manual	<code>\$bindable()</code>

Snippets repetidos entre componentes irmãos vão para um `.svelte` próprio e são exportados pelo `<script module>` — o que só funciona se o snippet não referenciar nada do `<script>` de instância, então os imports de que ele depende também ficam no bloco `module`.

22.2 SvelteKit — server-first

Devolva no `load()` **apenas o que a página consome**. Papéis do usuário não precisam ir no payload (a UI lê `page.data.usuario` via `useAutorizacao`), e parâmetros de URL usados só para montar a query ficam no servidor.

Prefira, nesta ordem:

1. Server Action (`+page.server.ts` com `actions`) para mutações de formulário;
2. `load()` no servidor para dados iniciais;
3. `invalidate()` / `invalidateAll()` para revalidar sem recarregar a página inteira.

22.3 O guia visual

Os tokens vivem em `src/theme.css` (paleta oklch de 7 canais) e `src/app.css`. Resumo das regras que mais aparecem em revisão de PR:

Tema	Regra
Cores	Sempre pelos canais (<code>primary</code> , <code>secondary</code> , <code>tertiary</code> , <code>success</code> , <code>warning</code> , <code>error</code> , <code>surface</code>). Nunca <code>text-red-500</code>
Texto pequeno	Só <code>text-2xs</code> (0,7rem) e <code>text-3xs</code> (0,625rem). Nada de <code>text-[...]</code> arbitrário
Contraste	Par padrão <code>text-surface-600 dark:text-surface-400</code> . <code>text-surface-500</code> não é permitido sobre fundo claro
Foco	Nunca <code>outline-none</code> sem substituto visível (<code>focus-visible:ring-2 ...</code>)
Modais	Use <code>\$lib/components/ModalShell.svelte</code> . Não reimplemente foco, Escape, ARIA ou scroll lock
Botões	CTA <code>preset-filled-primary-500</code> · destrutivo <code>preset-filled-error-500</code> · neutro <code>preset-outlined-surface-500</code>
Tamanho de botão	<code>py-1.5</code> (navegação) · <code>py-2.5</code> (CTA de modal) · <code>py-3.5</code> (ação final de página)
Voltar	<code>\$lib/components/BotaoVoltar.svelte</code> , sempre acima do <h1> , nunca no rodapé
Rodapé de ação	<code>\$lib/components/RodapeAcoes.svelte</code> (<code>sticky</code> , não <code>fixed</code>)
Border-radius	<code>--radius-base</code> (botões/inputs), <code>--radius-container</code> (cards/modais), <code>rounded-full</code> (pills)
Z-index	Escala fixa: <code>z-10</code> local · <code>z-40</code> topbar mobile · <code>z-50</code> sidebar/modais · <code>z-[60]</code> / <code>z-[70]</code> modal sobre modal · <code>z-[100]</code> diálogos globais · <code>9999</code> toasts
Ícones	<code>@lucide/svelte</code> . Nunca emoji como ícone — renderizam diferente por SO e ignoram o tema
Loading	<code>\$lib/components/Spinner.svelte</code> ; skeletons para página; <code>loading.show()/hide()</code> para API
Tabelas	<code><div class="hidden md:block table-wrap"><table class="table"></code> no desktop + cards <code>md:hidden</code> no mobile
Transições	Propriedade específica (<code>transition-colors</code>), não <code>transition-all</code>

22.4 A armadilha do Skeleton: `transform` não se anula com `translate-*`

Vale conhecer porque não gera erro nem aviso — só sai do lugar.

Os utilitários `translate- / scale- / rotate-*` do Tailwind v4 escrevem as **propriedades individuais** (`translate` , `scale` , `rotate`). O CSS de componente do Skeleton posiciona peças com a propriedade **`transform`** . São propriedades diferentes: o navegador aplica as duas e os deslocamentos **somam** .

Foi a causa única de dois "desalinhamentos": o thumb do `ToggleSwitch` saía 14 px fora do trilho, e o chevron do `SearchableSelect` vazava 7 px acima do campo.

Para neutralizar um `transform` do Skeleton, zere **`transform`** — e isso está resolvido de uma vez no `app.css` , por seletor, não classe por classe. Ao adotar um componente novo do Skeleton, confira o CSS dele antes de posicionar peças por classe.

22.5 Onde a regra deve morar

Três heurísticas que resolvem a maioria das dúvidas de "onde ponho isso":

1. Se precisa de teste, sai do `.svelte`. Componente não tem teste unitário aqui. Foi assim que nasceram `menu-visibility.ts`, `bem-vindo-cards.ts`, `status-escala.ts` e `mensagens-download.ts`.
2. Se dois lugares fazem a mesma pergunta, a resposta é uma função. Capítulo 12.
3. Se o valor sai impresso num documento, ele congela. Capítulo 19.6.

22.6 Documentação de código

A régua é `npm run docs:inventario`, com três alvos em ordem de retorno:

1. **cabeçalho de módulo** (no topo do arquivo, **antes dos imports**);
2. **contrato de export público** (o que devolve, o que assume, que efeito tem);
3. comentário de ponto em trecho opaco.

Comentário explica DECISÃO — regra da corporação, ordem obrigatória, armadilha de biblioteca, consequência legal —, nunca o que o código já diz. `/** Busca a escala por id. */` é dívida: ocupa espaço, envelhece e não diz nada que a assinatura já não diga.

⚠ Densidade de comentário **não é meta**. Um componente com 800 linhas de markup e 2% de comentário pode estar correto — o que falta nele é o cabeçalho. O único gate automático é para arquivo **novos** em `lib/db` (`npm run docs:guard`).

23. Testes

23.1 Os dois níveis

Nível	Ferramenta	Onde	Roda
Unitário	Vitest	<code>src/**/*.test.ts</code>	<code>npm run test</code>
Ponta a ponta	Playwright	<code>e2e/*.spec.ts</code>	<code>npm run test:e2e</code>

23.2 O que testar em cada nível

Vitest cobre regra pura: permissões, cálculos, parsing, criptografia, schemas, camada de dados. Os grupos principais:

Pasta	Cobre
<code>src/lib/__tests__</code>	Autenticação (PBKDF2/pepper, sessões, 2FA), CSRF, headers
<code>src/lib/schemas/__tests__</code>	Schemas Zod
<code>src/lib/gise/__tests__</code>	Regras GISE puras: etapas, renumeração, tipos de pergunta
<code>src/lib/crypto/__tests__</code>	Criptografia de campos e CPF, primitivas
<code>src/lib/db/__tests__</code>	Auditoria forense, retenção LGPD, upserts de assinatura
<code>src/lib/server/assinatura/__tests__</code>	CAdES, OCSP, TSA, trust store, ByteRange, goldens visuais
<code>src/lib/server/auth/__tests__</code>	Login, certificado, revogação, webhooks
<code>src/lib/server/export/__tests__</code>	Goldens de PDF e cabeçalho institucional

Playwright cobre o que só existe no navegador: foco, `inert`, media queries, view transitions, e os fluxos completos.

23.3 O spec que se escreve sozinho

`e2e/autorizacao-negativa.spec.ts` varre `src/routes/` **em tempo de teste e exercita** todas** as operações materiais em dois cenários (anônimo, e policial de outra unidade contra um recurso real).

É o complemento executável do `guard:autorizacao`: **o guard vê se existe gate; o spec vê se ele vem antes do trabalho e se olha o recurso, não só o usuário.**

23.4 O fluxo A3 em CI, sem token físico

O build de E2E injeta uma **CA de teste** no trust store ICP-Brasil (`E2E_TEST_CA=1` no build → `define` do Vite → `trust-store.ts`; chaves regeneradas a cada execução). O spec faz o papel do Assinador SERPRO e percorre preparar → finalizar → download → `/validar` contra a verificação real do servidor, incluindo os negativos (CA desconhecida, CPF divergente, digest adulterado).

 Em build normal a constante não existe e o ramo é código morto: **não há variável de runtime capaz de ligar a CA de teste em produção.** Quando você mexer nessa área, mantenha essa propriedade.

23.5 Escrevendo um teste unitário

```
// src/lib/gise/__tests__/horarios.test.ts
import { describe, it, expect } from 'vitest';
import { horarioEfetivo, temHorarioProprio } from '../horarios';

describe('horarioEfetivo', () => {
  it('herda da seccional quando a equipe não tem horário', () => {
    expect(horarioEfetivo({ equipe: null, seccional: '08:00', escala: '07:00'
  })).toBe('08:00');
  });

  it("trata '08' e '08:00' como a mesma hora", () => {
    expect(temHorarioProprio({ equipe: '08', seccional: '08:00' })).toBe(false);
  });
});
```

Boas práticas observadas no repositório:

- o nome do teste descreve a **regra**, não a função;
- casos de fronteira primeiro (nulo, vazio, limite);
- fixture lida por caminho fica em `__tests__/fixtures/`.

23.6 Testes manuais

`TESTING.md` é o roteiro de **exceção**: cobre o que exige hardware ou ambiente real (Assinador SERPRO com token físico, caixa de e-mail, autoridade de carimbo de tempo). O gate de regressão é a suíte automatizada; os casos já cobertos por spec estão anotados no próprio arquivo.

24. CI, guards e deploy

24.1 O que o CI roda

Todo PR para `main` ou `staging` dispara `.github/workflows/deploy.yml`:

#	Passo	Falha significa
1	<code>npm ci + npx svelte-kit sync</code>	—
2	<code>npm run lint:ci (--max-warnings 0)</code>	Warning novo de ESLint
3	<code>npm run format:check e</code> <code>format:check:e2e</code>	Rode <code>npm run format / format:e2e</code>
4	<code>npm run knip</code>	Código ou export morto
5	<code>npx svelte-check --threshold error</code>	Erro de tipo
6	<code>npx vitest run --coverage</code>	Teste unitário quebrado
7	<code>npm run build</code>	Build quebrado
8	Guard — convenção de testes	<code>*.test.ts</code> fora de <code>__tests__</code>
9	Guard — padrão de erros API	<code>return json({ error</code> em <code>src/routes/api</code>
10	Guard — permissão de documento assinado	Falta importar <code>verificarPermissaoEscala / Gise</code> nas rotas críticas
11	<code>guard:autorizacao</code>	Operação material que não recusa ninguém
12	<code>guard:duplicacao</code>	Bloco de 10 linhas repetido
13	<code>guard:achados</code>	Sigla citada no código sem catálogo
14	<code>guard:entrada</code>	<code>FormData</code> lido sem limite
15	<code>docs:guard</code>	Arquivo novo em <code>lib/db</code> sem cabeçalho/JSDoc
16	Migrações D1 locais + <code>npx playwright test</code>	E2E quebrado
17	<code>wrangler pages deploy</code>	Deploy



Rode os guards localmente **antes** de abrir o PR — todos existem como script npm. Descobrir no CI o que um `npm run guard:duplicacao` de 5 segundos diria é desperdício de ciclo.

24.2 Duas decisões do workflow que valem entender

cancel-in-progress só em **pull request**, nunca em **push**. Num push para `main`, dois merges em sequência colocariam os dois runs no mesmo grupo e o segundo cancelaria o primeiro — que é justamente o que faz

`db:migrate:prod` e `pages deploy`. O `migrate.ts` grava em `_migrations_aplicadas` **depois** do `dl execute`, então um cancelamento nessa janela deixa a migração aplicada e não registrada.

timeout-minutes: 30. Existe porque um step do Playwright travou no `apt-get update` por horas, e tudo em `main` enfileirou atrás dele. Se um dia estourar por crescimento honesto da suíte, o lugar de olhar é o passo de E2E — **suba o número, não remova**.

24.3 Deploy

Recomendado: push/PR para `main` ou `staging`, e o CI faz tudo.

Manual:

```
npm run db:migrate:prod -- --yes    # SEMPRE antes de deployar código novo
npm ci
npm run build
wrangler pages deploy .svelte-kit/cloudflare --project-name=escalas
```

Checklist pré-deploy:

- ☐ Variáveis de ambiente configuradas no dashboard Cloudflare
- ☐ `RESET_TOKEN` diferente de `SYNC_TOKEN` (ou intencionalmente vazio)
- ☐ Migrações aplicadas
- ☐ `npm run test verde`
- ☐ `npm run check limpo`
- ☐ `npm run build` sem erros

Smoke tests pós-deploy: acessar e logar; `GET /api/health` → 200; abrir uma escala e conferir o PDF; olhar os logs em Cloudflare Pages → Functions.

24.4 O endpoint destrutivo

`/api/webhook/reset-policiais` **apaga todas as tabelas operacionais**. Exige quatro camadas simultâneas:

1. Authorization: Bearer `<SYNC_TOKEN>`
2. X-Reset-Token: `<RESET_TOKEN>`
3. X-Confirm-Reset: `<YYYY-MM-DD em UTC>` — confirmação explícita do dia
4. X-Webhook-Timestamp + X-Webhook-Nonce — anti-replay de 5 min, **obrigatórios aqui** independentemente da configuração global, porque a camada 3 sozinha deixaria 24 h de janela para reenvio

⚠ Use apenas pelo menu da planilha Google, que exige confirmação dupla.

25. Sua primeira contribuição

Este capítulo é um passo a passo completo. O exemplo é fictício mas realista: **adicionar um campo "telefone de contato da equipe" no plano operacional, exibido na tela e impresso no Anexo I do PDF**.

25.1 Antes de escrever código

1. **Entenda a regra de negócio.** Pergunte: o campo é obrigatório? Tem limite de tamanho? Sai impresso? Quem pode editar?
2. **Ache o módulo.** `/gise/planos/[id]` → `src/routes/gise/planos/[id]/`.
3. **Leia o `+page.server.ts` inteiro** antes de mudar qualquer coisa.
4. **Rode os testes** para saber o que já está verde: `npm run test`.

25.2 O caminho completo, na ordem

Passo 1 — Migração.

```
-- migrations/0075_plano_equipe_telefone.sql
ALTER TABLE plano_equipas ADD COLUMN telefone_contato TEXT NOT NULL DEFAULT '';
```

```
npm run db:migrate
```

Passo 2 — Schema Drizzle (`src/lib/server/schema.ts`), na tabela `planoEquipes` :

```
telefone_contato: text('telefone_contato').notNull().default(''),
```

⚠ Os passos 1 e 2 são **um par**. Fazer só o 2 compila e quebra em runtime (§7.3).

Passo 3 — Camada de dados (`src/lib/db/planos/...`): inclua a coluna no `select` e no `update`. Se o arquivo for novo, ele precisa de cabeçalho e JSDoc — o CI verifica.

Passo 4 — Limite de entrada. É form action, então use o leitor com limite:

```
import { textoLimitado } from '$lib/server/form-data';
const MAX_TELEFONE = 20;
const telefone = textoLimitado(fd, 'telefone_contato', MAX_TELEFONE);
```

E use **o mesmo número** no `maxlength` da tela (§11.3).

Passo 5 — Autorização. A action já tem portão? Confirme que ela recusa alguém e que o portão olha **o recurso** (o plano), não só o usuário. Se você criou uma action nova, ela precisa do seu próprio gate.

Passo 6 — Tela. No `.svelte`, campo controlado por `$state`, `maxlength` igual ao servidor, rótulo e classe seguindo o guia visual (§22.3).

Passo 7 — PDF. Aqui mora o ritual dos goldens:

```
npm run test -- pdf-goldens      # 1. confirme verde ANTES
# ... faça a alteração no gerador do Anexo I ...
npm run test -- pdf-goldens      # 2. vai falhar: a saída mudou
UPDATE_PDF_GOLDENS=1 npm run test -- pdf-goldens  # 3. só se a mudança é INTENCIONAL
git diff                        # 4. confira o golden gerado antes de commitar
```

Passo 8 — Testes. Regra nova em `.ts` puro ganha teste unitário em `__tests__`/. Fluxo de tela ganha (ou estende) um spec em `e2e/`.

Passo 9 — Documentação viva. Se o comportamento mudou, `README.md` muda **no mesmo PR**. Variável nova entra em `.env.example`. Passo operacional novo entra em `DEPLOY.md`.

Passo 10 — Verificação local.

```
npm run lint:fix && npm run format
npm run check
npm run test
npm run guard:autorizacao && npm run guard:duplicacao && npm run guard:entrada && npm run guard:achados
```

25.3 Checklist de PR

- ☐ Migração e schema, se o banco mudou
- ☐ Entrada limitada no servidor (Zod ou `form-data`)
- ☐ Operação material recusa alguém, e o portão olha o recurso
- ☐ Erros pelos helpers de `$lib/server/api`
- ☐ Fetch no cliente por `$lib/api-fetch`
- ☐ Nada de lógica duplicada — extraiu em vez de comentar
- ☐ Nenhum dado pessoal a mais no payload do `load`
- ☐ Goldens conferidos, se tocou em PDF/e-mail
- ☐ Teste unitário para regra nova; spec E2E para fluxo novo
- ☐ Documento vivo atualizado no mesmo PR
- ☐ `npm run check`, `test` e os guards verdes localmente

25.4 Como pedir ajuda de forma eficiente

Traga três coisas: **a rota, o arquivo e o que você já tentou**. "O download da escala dá 403 para o admin de seccional; olhei `verificarPermissaoEscala` e o escopo parece cobrir a lotação; não achei onde o portão recusa" é uma pergunta que se responde em dois minutos. "Não está funcionando" não é.

26. Exercícios guiados

Faça na ordem. Cada um tem uma dica e o lugar onde a resposta se confirma. Nenhum exige alterar o sistema — são exercícios de **leitura**, que é a habilidade que falta a quem chega.

Exercício 1 — Do clique ao banco.

Abra `/unidades` e acompanhe o que acontece ao salvar uma unidade nova.

Dica: comece por `src/routes/unidades/+page.server.ts`.

Perguntas: qual função valida o formulário? O que acontece se o nome já existir? Por que não existe uma action de excluir?

Exercício 2 — O portão.

Encontre, em `verificarPermissaoEscala`, o caso em que um admin **de fora** do escopo ainda assim pode ler a escala.

Perguntas: que tabela sustenta essa exceção? Por que ela existe?

Exercício 3 — A ordem importa.

Leia o cabeçalho de `src/hooks.server.ts`.

Perguntas: o que quebraria se `handleAuth` viesse antes de `handleCsrf`? E se `handleRequestContext` não fosse o primeiro?

Exercício 4 — Encontre a duplicação.

Rode `npm run guard:duplicacao` e abra `scripts/duplicacao-baseline.json`.

Perguntas: escolha uma entrada e leia o campo `nota`. Você concorda com a decisão de manter? O que seria preciso para extraí-la?

Exercício 5 — A entrada.

Escolha uma form action qualquer em `src/routes/`.

Perguntas: todo campo lido do `FormData` tem limite? Se algum não tem, por que o `guard:entrada` não reclama? (Resposta possível: o campo é identificador, booleano, ou está numa das cinco formas aceitas — §11.2.)

Exercício 6 — A minimização.

Abra o `load` de `src/routes/gise/[id]/+page.server.ts`.

Perguntas: que campos de presença chegam ao cliente? Onde está `presencasParaCliente` e por que ele monta um objeto novo em vez de deletar chaves?

Exercício 7 — O documento.

Encontre onde o manifesto do PDF decide entre "QUALIFICADA · ICP-BRASIL" e o selo institucional.

Dica: o critério é um campo do banco, não a existência de uma linha (§14.1).

Exercício 8 — O teste que se escreve sozinho.

Leia `e2e/autorizacao-negativa.spec.ts`.

Perguntas: como ele descobre as rotas? Por que ele não pode confiar no status HTTP das form actions?

Exercício 9 — A conta.

Em `$lib/diarias/`, siga o caminho de uma equipe que sai às 04:00 e percorre 150 km numa segunda-feira.

Perguntas: qual rubrica sai? O que muda se a operação durar 3 horas em vez de 4?

Exercício 10 — Sua primeira alteração de verdade.

Escolha uma pendência pequena com o time (um rótulo, uma validação, um teste faltando) e siga o capítulo 25 de ponta a ponta, incluindo o checklist de PR.

27. Troubleshooting e FAQ

27.1 Ambiente

Sintoma	Causa	Solução
Erro de binding D1/R2 no <code>dev</code>	Banco local não existe	<code>npm run db:migrate</code>
Type error só no <code>check</code> , não no editor	<code>svelte-check</code> é mais rigoroso	Confie no <code>check</code>
CSRF 403 em chamada de API	<code>fetch</code> cru	Use <code>apiFetch</code>
Sessão expira logo após login	Relógio dessincronizado	Ajuste o NTP
Specs de webhook com 401	<code>SYNC_TOKEN</code> curto	<code>openssl rand -hex 32</code>
<code>SELECT</code> falha com coluna inexistente	Schema editado sem migração	Escreva a migração e rode <code>db:migrate</code>
Busca devolve 500	Padrão <code>LIKE</code> acima de 50 chars	Use <code>likeContains</code> , não <code>like()</code>

27.2 Assinatura

Sintoma	Causa provável
Assinador SERPRO não conecta	Aplicativo fechado, ou certificado self-signed não aceito. Abra <code>https://assinador-desktop.serpro.gov.br:65166</code> e aceite
<code>finalizar</code> recusa com conflito	Intenção já usada, alvo divergente ou hash do PDF diferente do preparado
<code>/validar</code> diz "cadeia indisponível"	Trust store ICP-Brasil vazio; ver o README da pasta <code>icp-brasil/</code>
Manifesto sem GPS	Coordenada inválida persistida como ausência (§11.5)

27.3 Perguntas frequentes de quem chega

"Posso usar `console.log`?"

Para depurar local, sim. No código que vai para o PR, use `logger` — ele carrega o contexto da requisição e é o que alimenta `/auditoria/logs`.

"Posso instalar uma biblioteca nova?"

Pergunte antes. O bundle importa (o sistema é usado no celular, em rede ruim), e dependências pesadas precisam entrar no code splitting de `vite.config.ts`. Além disso, tudo precisa funcionar no runtime da Cloudflare — bibliotecas que dependem de `fs`, `net` ou APIs de Node não funcionam.

"Achei um comentário enorme explicando uma decisão. Posso apagar?"

Não. Esses comentários são o registro do incidente que criou a regra. Se a decisão mudou, atualize o comentário junto com o código.

"O guard está reprovando meu PR e eu não entendo por quê."

Cada guard imprime o motivo e o arquivo. Se for `duplicacao`, extraia; se for `entrada`, limite; se for `autorizacao`, adicione o gate ou declare a dispensa **com o motivo**. Regravar baseline não é solução (§11.4 e

§12.3).

"Preciso mesmo escrever teste?"

Regra nova em `.ts` puro: sim. Fluxo novo de tela: um spec E2E. Ajuste de markup: não necessariamente.

"Como eu sei se algo é 'operação material'?"

Se muda estado no banco ou no R2, é. `POST`, `PUT`, `PATCH`, `DELETE` e toda form action.

"Onde eu leio mais sobre X?"

Apêndice E tem o mapa. Em geral: comportamento → `README.md`; operação/produção → `DEPLOY.md`; regra de código → `CLAUDE.md`; histórico de uma decisão → `docs/HISTORICO.md`.

Apêndices

Apêndice A — Inventário de rotas

A.1 Páginas

Rota	Módulo
/	Raiz (redireciona conforme papel)
/login	Autenticação
/alterar-senha, /redefinir-senha, /aceitar-termo	Onboarding e recuperação
/bem-vindo, /escalas/bem-vindo, /gise/bem-vindo, /super-admin	Telas de boas-vindas por perfil
/escalas, /escalas/nova, /escalas/[id]	Escalas
/painel, /recebidos	Consoles de escala
/gise, /gise/[id], /gise/finalizadas, /gise/operacoes	Escala extra
/gise/planos, /gise/planos/novo, /gise/planos/[id]	Plano operacional
/res-gise, /res-gise/relatorio/[giseId]	Visão do membro
/dados-base, /dados-base/[operacaoId]	Linha de base dos indicadores
/produtividade	Painel de produtividade
/policiais, /policiais/[id], /policiais/upload	Cadastro de servidores
/unidades	Unidades
/solicitacoes	Fila de decisão
/perfil	Meu perfil
/conf-ass, /config-geral, /config-custos	Configurações
/auditoria, /auditoria/logs	Consoles forenses
/validar, /validar/[hash]	Validação pública
/termo/[versao], /termo/dpo	Termo de uso (público)

A.2 Endpoints por grupo

Grupo	Endpoints
auth	login, logout, verificar-2fa, reenviar-codigo, solicitar-redefinicao, confirmar-redefinicao, certificado/iniciar, certificado/verificar, alternar-acesso, alternar-modulo, reautenticar-assinatura, solicitar-codigo-assinatura, solicitar-verificacao-email-pessoal, confirmar-verificacao-email-pessoal
escalas/[id]	download, documento-assinado (GET/DELETE), preparar-assinatura, finalizar-assinatura, preparar-assinatura-avancada, finalizar-assinatura-avancada, assinar-simples, solicitar-assinatura (POST/DELETE)
gise/[id]	download, documento-assinado, documento-assinado/info, finalizar, reabrir, impacto-exclusao, os quatro preparar/finalizar de assinatura, presenca/ (4), relatorios/[seccionalId]/ (5)
planos	[id]/download
policiais	search, [id]/passkey, [id]/email-aviso, historico/[eventoId]/documento, solicitacoes/[solicitacaoId]/documento
produtividade	ordem (PUT)
admin	audit, compliance, lgpd/incidentes, lgpd/incidentes/[id], lgpd/solicitacoes, lgpd/solicitacoes/[id], lgpd/limpeza
lgpd	solicitar
webauthn	registro (GET/POST/DELETE), solicitar-codigo-reposicao
webhook	sync-policiais, sync-unidades, reset-policiais, limpeza-retencao
Público	validar/[hash]/download, validar/logo, health
Outros	configuracoes/assinatura (GET/PUT), unidades/search, sync/estado, gise/historico/export

Apêndice B — Tabelas do banco

Tabela	Conteúdo
<code>policiais</code>	Servidores: matrícula, CPF cifrado, cargo, classe, lotação, senha PBKDF2, papel
<code>administradores</code>	Contas de administrador (e o vínculo com <code>policiais</code>)
<code>sessoes</code>	Sessões ativas (token, tipo, expiração)
<code>aceites_termos</code>	Aceites de termo (versão, hash, IP, user-agent)
<code>credenciais_webauthn</code> , <code>webauthn_desafios</code> , <code>passkey_reposicao</code>	Chave de assinatura
<code>unidades</code>	Hierarquia departamento → seccional → delegacia
<code>policial_historico</code>	Histórico funcional, com PDF no R2
<code>cadastro_solicitacoes</code>	Pedidos de correção de campo
<code>policial_acao_solicitacoes</code>	Pedidos de movimentação/afastamento/desvinculação
<code>escalas</code> , <code>escala_policiais</code>	Escalas e alocação
<code>escala_documentos</code>	PDF assinado com metadados CAdES-LT
<code>escala_solicitacoes_assinatura</code>	Solicitações de assinatura
<code>operacoes</code> , <code>operacao_linha_base</code>	Catálogo de operações e denominadores das metas
<code>gise_escalas</code> , <code>gise_seccionais</code> , <code>gise_equipes</code> , <code>gise_membros</code> , <code>gise_seccional_unidades</code>	Estrutura da escala extra
<code>gise_presencas</code> , <code>gise_presenca_termos</code>	Presença e termos
<code>gise_documentos</code> , <code>gise_assinaturas_relatorios</code>	Documentos e assinaturas de relatório
<code>gise_modelo_formulario</code> , <code>gise_respostas_formulario</code>	Formulário e respostas
<code>custo_parametros</code>	Valores de hora extra e diária, em centavos (append-only)
<code>planos_operacionais</code> , <code>plano_equipes</code> , <code>plano_equipe_membros</code> , <code>plano_opcoes</code>	Plano operacional
<code>municipios</code> , <code>distancias_municipios</code> , <code>distancias_medicao</code>	Matriz rodoviária
<code>assinatura_intencoes</code> , <code>assinatura_reauth</code>	Intenção de uso único e reautenticação
<code>configuracoes</code>	Flags (incluindo política de assinatura)
<code>audit_log</code> , <code>audit_pendencias</code> , <code>audit_checkpoints</code>	Trilha forense
<code>app_log</code>	Logs técnicos
<code>lgpd_incidentes</code> , <code>lgpd_solicitacoes</code>	Conformidade
<code>login_attempts</code> , <code>recovery_attempts</code> , <code>webhook_nonces</code>	Limites e anti-replay

Tabela	Conteúdo
r2_pendencias, base_equipe_pendencias, dois_fatores_tokens, reset_senha_tokens	Filas e tokens

Apêndice C — Variáveis e bindings

C.1 Bindings Cloudflare (wrangler.toml)

Binding	Tipo	Uso
escalas_db	D1	Banco principal
escalas_docs	R2	PDFs, selfies, documentos
EMAIL	Email Sending	Envio primário de e-mail

C.2 Variáveis principais

Variável	Obrigatória	Papel
SYNC_TOKEN	✓	Bearer dos webhooks de sincronização (mín. 32 chars)
RESET_TOKEN	⚠	Segredo separado do endpoint destrutivo. Ausente = 401
RESEND_API_KEY + RESEND_FROM_EMAIL	✓ prod	Fallback de e-mail. Sem e-mail, o 2FA trava
PASSWORD_PEPPER	⚠ prod	HMAC antes do PBKDF2. Nunca rotacionar sem plano
CPF_ENCRYPTION_KEY	⚠ prod	AES-256-GCM do CPF em repouso
CPF_INDEX_KEY	⚠ prod	Índice cego para lookup sem decifrar
RATE_LIMIT_IP_SALT	⚠ prod	Evita lockout de NAT corporativo
APP_ORIGIN	⚠ prod	Origem canônica nos links de e-mail
SENTRY_DSN, SENTRY_ENVIRONMENT	✗	Monitoramento
ICP_BRASIL_TRUST_STORE_REQUIRED	✗	Trust store vazio vira erro duro
TSA_URL, TSA_USERNAME, TSA_PASSWORD	✗	Carimbo de tempo
EXIGIR_TSA_QUALIFICADA	✗	Recusa assinatura sem carimbo de ACT credenciada
SUPER_ADMIN_, ADMIN_GERAL_	✗	Contas de bootstrap (logins auditados)
GISE_BASE_EQUIPE_WEBHOOK_URL / _SECRET	✗	Integração com a planilha

📖 A fonte autoritativa é `.env.example`; os tipos estão em `src/app.d.ts`; o detalhe operacional está em `DEPLOY.md`.

Apêndice D — Comandos

```
# Desenvolvimento
npm run dev                # servidor local (5173)
npm run build              # build de produção
npm run preview            # pré-visualização da build

# Qualidade
npm run check              # type-check (Svelte Check + TS)
npm run check:watch
npm run lint / lint:fix / lint:strict / lint:ci
npm run format / format:check / format:e2e / format:check:e2e
npm run knip               # código e exports mortos

# Guards (todos rodam no CI)
npm run guard:autorizacao
npm run guard:duplicacao
npm run guard:achados
npm run guard:entrada
npm run docs:guard
npm run docs:inventario    # inventário de documentação (priorização)

# Testes
npm run test / test:watch / test:coverage
npm run test:e2e / test:e2e:ui / test:e2e:report

# Banco
npm run db:migrate
npm run db:migrate:staging
npm run db:migrate:prod -- --yes

# Utilitários de usuários (exigem --yes; contra produção, CONFIRMO_PRODUCAO)
npm run users:set-default-password -- --yes
npm run users:clear-passwords-non-admins -- --yes
```

Apêndice E — Onde ler mais

Documento	Quando abrir
<code>README.md</code>	Visão geral, setup, arquitetura, módulos, troubleshooting
<code>CLAUDE.md</code>	As regras obrigatórias de código. Leia inteiro no primeiro dia
<code>DEPLOY.md</code>	Runbook de produção: secrets, papéis, backup/rollback, trust store, go-live
<code>TESTING.md</code>	Roteiro manual de exceção (hardware, e-mail real)
<code>.env.example</code>	Fonte autoritativa das variáveis
<code>docs/README.md</code>	Índice da documentação (vivo × histórico)
<code>docs/HISTORICO.md</code>	Catálogo das auditorias arquivadas e o <code>git show</code> de cada uma
<code>docs/QA_ASSINATURA_A3_DESKTOP.md</code>	QA manual do fluxo Token A3
<code>scripts/README.md</code>	Scripts utilitários e integração Google Sheets
<code>src/lib/server/assinatura/icp-brasil/README.md</code>	Trust store ICP-Brasil
Svelte 5	< https://svelte.dev/docs/svelte/overview >
SvelteKit	< https://svelte.dev/docs/kit/introduction >
Drizzle ORM	< https://orm.drizzle.team >
Cloudflare D1	< https://developers.cloudflare.com/d1/ >
Zod	< https://zod.dev >

Palavra final

Este sistema é lido muito mais vezes do que é escrito, e quase sempre por alguém com pressa resolvendo um problema em produção. Cada regra desta apostila existe para que essa pessoa — que pode ser você daqui a seis meses — encontre o código dizendo a verdade sobre si mesmo.

Quando estiver em dúvida entre o caminho rápido e o caminho que deixa a decisão escrita, escolha o segundo. É o que o time faz aqui, e é por isso que o repositório ainda é navegável depois de 152 mil linhas.